```
import xarray as xr
import cfgrib
import numpy as np
import pandas as pd
import xarray as xr
import matplotlib.pyplot as plt

from scipy.stats import linregress


CFGRIB_KW = {"engine": "cfgrib", "backend_kwargs": {"indexpath": ""}}

def to_month_end(time_index):
    ts = pd.to_datetime(time_index)
    return ts.to_period("M").to_timestamp("M")

def clean_drop(ds):
    drop_vars = [v for v in ["step", "valid_time", "number",
"surface"] if v in ds.variables]
    return ds.drop_vars(drop_vars)

SHORTNAME_ALIASES = {
    "u10": ["u10", "10u"],
    "v10": ["v10", "10v"],
    "t2m": ["t2m", "2t"],
    "d2m": ["d2m", "2d"],
    "msl": ["msl"],
    "tcc": ["tcc"],
    "tp":  ["tp"],
    "ssrd":["ssrd"],
    "e":   ["e"],
}

def open_by_any_shortname(grib_path, aliases):
    """Próbuje po kolei shortName z listy; zwraca (ds,
użyty_shortName)."""
    last_err = None
    for sn in aliases:
        try:
            ds = xr.open_dataset(grib_path, **CFGRIB_KW,
filter_by_keys={"shortName": sn})
            if len(ds.data_vars) > 0:
                return ds, sn
        except Exception as e:
            last_err = e
    raise ValueError(f"Nie znaleziono żadnego z shortName={aliases} w
pliku {grib_path}. Ostatni błąd: {last_err}")

def pick_time_coord(ds):
    if "time" in ds.coords:
```

```
        return ds["time"].values
    if "valid_time" in ds.coords:
        return ds["valid_time"].values
    raise KeyError("Dataset nie ma ani 'time' ani 'valid_time' w
coords.")

def area_mean(da, lat_slice, lon_slice):
    """
    Oblicza średnią obszarową (cos(lat)-weighted) dla DataArray.
    """
    da_sel = da.sel(latitude=lat_slice, longitude=lon_slice)

    # wagi szerokości geograficznej
    weights = np.cos(np.deg2rad(da_sel.latitude))
    weights.name = "weights"

    return da_sel.weighted(weights).mean(dim=("latitude",
"longitude"))


MONTHLY_GRIB = "data_month1.grib"

wanted_inst = ["u10", "v10", "d2m", "t2m", "msl", "tcc"]
wanted_acc  = ["tp", "ssrd", "e"]

inst_list = []
for name in wanted_inst:
    ds_one, used_sn = open_by_any_shortname(MONTHLY_GRIB,
SHORTNAME_ALIASES[name])
    ds_one = clean_drop(ds_one)

    ds_one =
ds_one.assign_coords(time=to_month_end(pick_time_coord(ds_one)))
    inst_list.append(ds_one)

acc_list = []
for name in wanted_acc:
    ds_one, used_sn = open_by_any_shortname(MONTHLY_GRIB,
SHORTNAME_ALIASES[name])
    ds_one = clean_drop(ds_one)
    ds_one =
ds_one.assign_coords(time=to_month_end(pick_time_coord(ds_one)))
    acc_list.append(ds_one)

ds_inst = xr.merge(inst_list, compat="override")
ds_acc  = xr.merge(acc_list,  compat="override")

ds_m = xr.merge([ds_inst, ds_acc], join="inner",
compat="no_conflicts").sortby("time")
```

```
ds_m["t2m"] = ds_m["t2m"] - 273.15
ds_m["d2m"] = ds_m["d2m"] - 273.15
ds_m["msl"] = ds_m["msl"] / 100.0
ds_m["tcc"] = ds_m["tcc"] * 100.0
ds_m["tp"]  = ds_m["tp"] * 1000.0
ds_m["e"]   = ds_m["e"] * 1000.0

ds_m["wind_speed"] = np.sqrt(ds_m["u10"]**2 + ds_m["v10"]**2)

ds_m
```

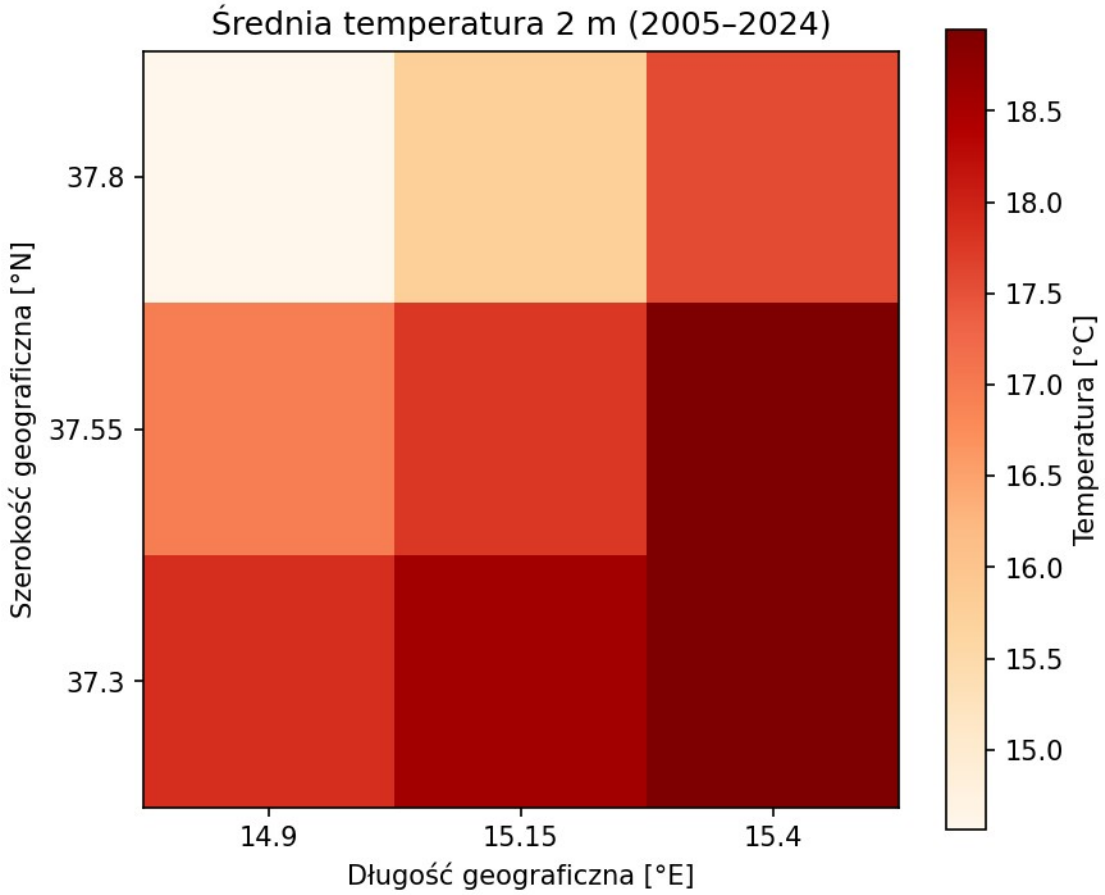
```
<xarray.Dataset> Size: 88kB
Dimensions:     (latitude: 3, longitude: 3, time: 239)
Coordinates:
  * latitude    (latitude) float64 24B 37.8 37.55 37.3
  * longitude   (longitude) float64 24B 14.9 15.15 15.4
  * time        (time) datetime64[ns] 2kB 2005-01-31 2005-02-28 ...
2024-11-30
Data variables:
    u10         (time, latitude, longitude) float32 9kB 0.9751
1.135 ... 0.1778
    v10         (time, latitude, longitude) float32 9kB -0.9285 ... -
0.4188
    d2m         (time, latitude, longitude) float32 9kB 1.569
3.109 ... 13.36
    t2m         (time, latitude, longitude) float32 9kB 4.823
7.067 ... 18.5
    msl         (time, latitude, longitude) float32 9kB 1.019e+03 ...
1.021e+03
    tcc         (time, latitude, longitude) float32 9kB 45.82
37.56 ... 35.47
    tp          (time, latitude, longitude) float32 9kB 3.563
4.445 ... 0.9452
    ssrd        (time, latitude, longitude) float32 9kB 1.159e+07 ...
8.249e+06
    e           (time, latitude, longitude) float32 9kB -1.223 -
1.714 ... -3.94
    wind_speed  (time, latitude, longitude) float32 9kB 1.346
1.811 ... 0.455
Attributes:
    GRIB_edition:            1
    GRIB_centre:             ecmf
    GRIB_centreDescription:  European Centre for Medium-Range Weather
Forecasts
    GRIB_subCentre:          0
    Conventions:             CF-1.7
    institution:             European Centre for Medium-Range Weather
Forecasts
    history:                 2026-01-05T10:57 GRIB to CDM+CF via
cfgrib-0.9.1...
```

```
ds_m.isnull().sum()
```

```
<xarray.Dataset> Size: 36B
Dimensions:  ()
Data variables:
    u10      int32 4B 0
    v10      int32 4B 0
    d2m      int32 4B 0
    t2m      int32 4B 0
    msl      int32 4B 0
    tcc      int32 4B 0
    tp       int32 4B 0
    ssrd     int32 4B 0
    e        int32 4B 0
Attributes:
    GRIB_edition:            1
    GRIB_centre:             ecmf
    GRIB_centreDescription:  European Centre for Medium-Range Weather
Forecasts
    GRIB_subCentre:          0
    Conventions:             CF-1.7
    institution:             European Centre for Medium-Range Weather
Forecasts
    history:                 2026-01-05T10:57 GRIB to CDM+CF via
cfgrib-0.9.1...
```

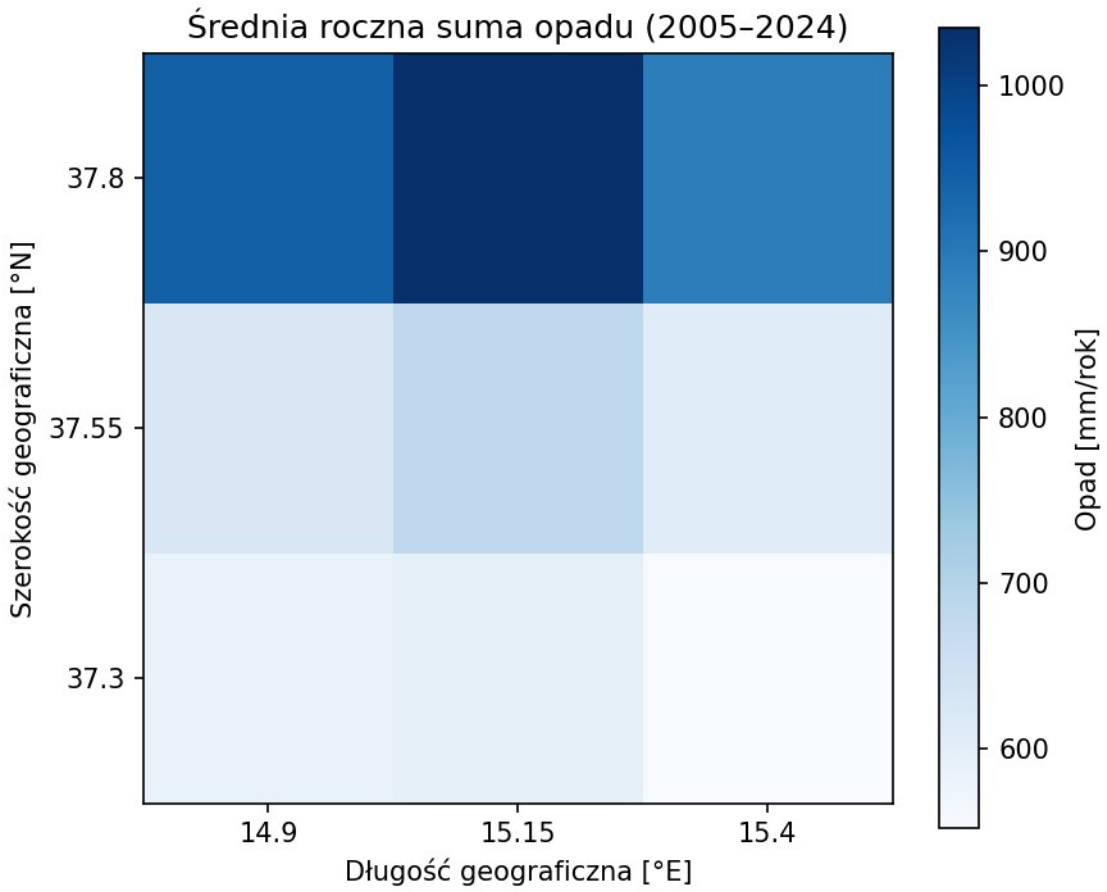
```
t2m_field = ds_m["t2m"].sel(
    time=slice("2005-01-01", "2024-12-31")
).mean(dim="time")
fig, ax = plt.subplots(figsize=(6,5), dpi=150)

im = ax.imshow(
    t2m_field.values,
    origin="upper",
    cmap="OrRd"
)

ax.set_xticks(np.arange(len(t2m_field.longitude)))
ax.set_xticklabels(np.round(t2m_field.longitude.values, 2))
ax.set_yticks(np.arange(len(t2m_field.latitude)))
ax.set_yticklabels(np.round(t2m_field.latitude.values, 2))

ax.set_xlabel("Długość geograficzna [°E]")
ax.set_ylabel("Szerokość geograficzna [°N]")
ax.set_title("Średnia temperatura 2 m (2005–2024)")

cbar = fig.colorbar(im, ax=ax)
cbar.set_label("Temperatura [°C]")
```

```
plt.tight_layout()
plt.show()
```



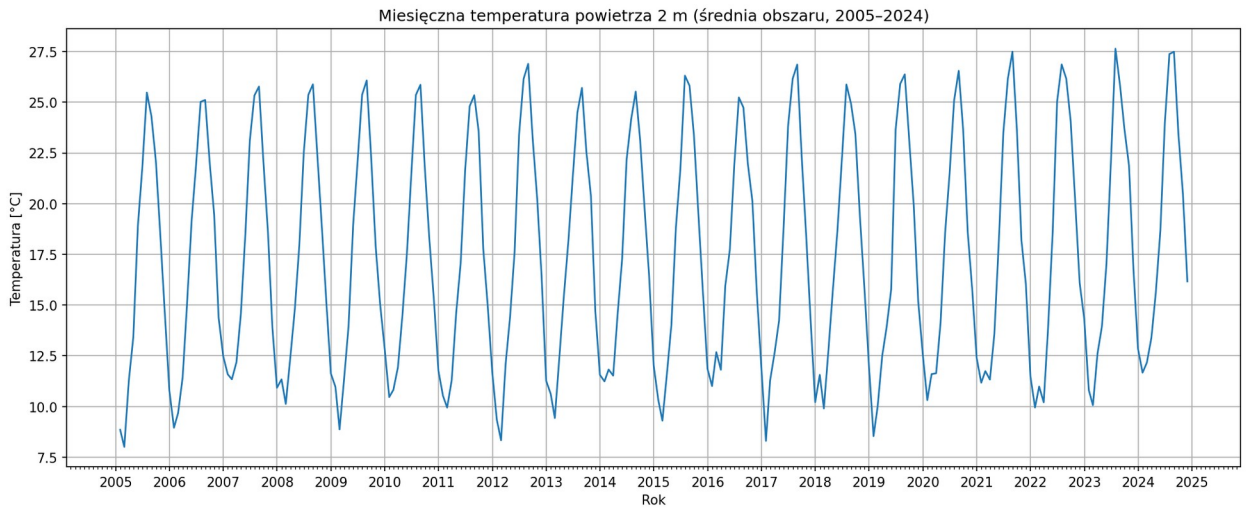

```
tp_daily_mean = ds_m["tp"]                      # mm/dzień (średnio w
miesiącu)
days = tp_daily_mean["time"].dt.days_in_month

tp_month_sum = tp_daily_mean * days             # mm/miesiąc
tp_year = tp_month_sum.resample(time="YE").sum("time")  # mm/rok
tp_clim = tp_year.mean("time")                   # mm/rok (2005–2024)
fig, ax = plt.subplots(figsize=(6,5), dpi=150)

im = ax.imshow(
    tp_clim.values,
    origin="upper",
    cmap="Blues",
    aspect="equal"
)

ax.set_xticks(np.arange(tp_clim.sizes["longitude"]))
ax.set_xticklabels(np.round(tp_clim.longitude.values, 2))
```

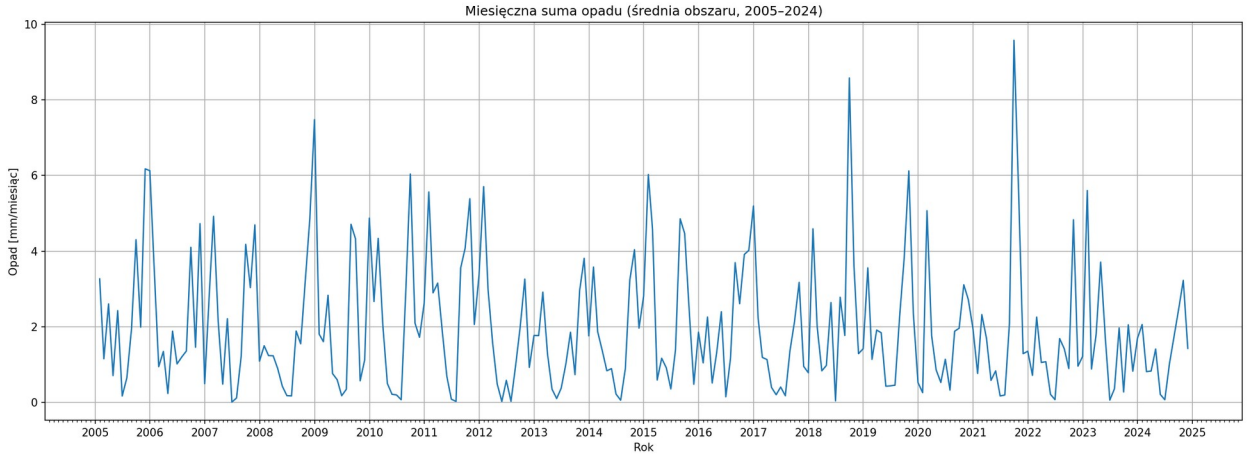
```
ax.set_yticks(np.arange(tp_clim.sizes["latitude"]))
ax.set_yticklabels(np.round(tp_clim.latitude.values, 2))

ax.set_xlabel("Długość geograficzna [°E]")
ax.set_ylabel("Szerokość geograficzna [°N]")
ax.set_title("Średnia roczna suma opadu (2005–2024)")

cbar = fig.colorbar(im, ax=ax)
cbar.set_label("Opad [mm/rok]")

plt.tight_layout()
plt.show()
```



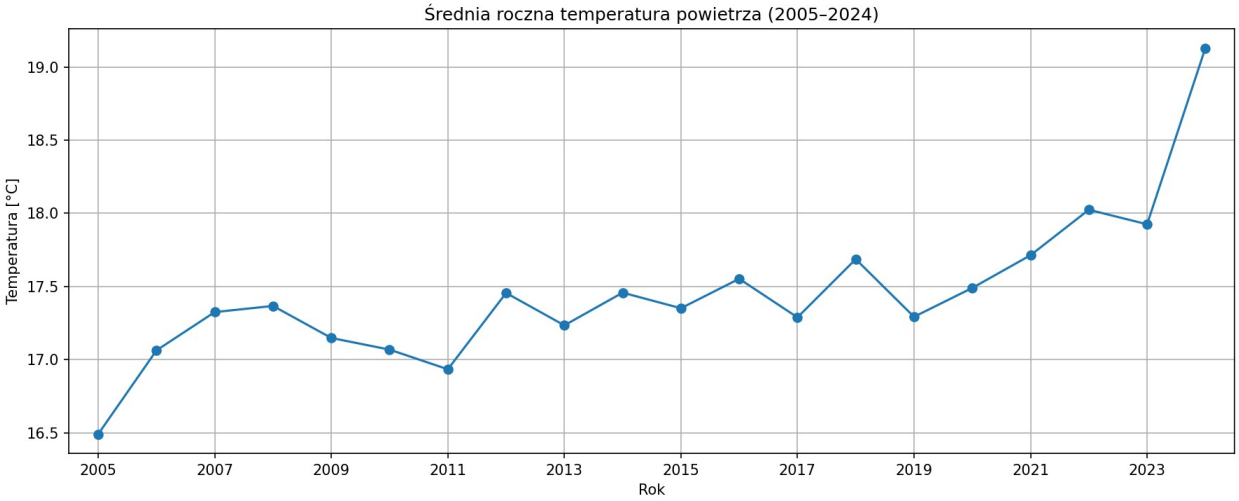

```
import matplotlib.dates as mdates
lat_slice = slice(37.8, 37.3)
lon_slice = slice(14.9, 15.4)

t2m_reg = area_mean(ds_m["t2m"], lat_slice, lon_slice)
tp_reg  = area_mean(ds_m["tp"],  lat_slice, lon_slice)

fig, ax = plt.subplots(figsize=(16,6), dpi=150)
```

```
ax.plot(t2m_reg["time"].values, t2m_reg.values, linewidth=1.3)

ax.set_title("Miesięczna temperatura powietrza 2 m (średnia obszaru,
2005–2024)")
ax.set_xlabel("Rok")
ax.set_ylabel("Temperatura [°C]")
ax.grid(True)
ax.xaxis.set_major_locator(mdates.YearLocator(base=1))
ax.xaxis.set_minor_locator(mdates.MonthLocator())
ax.xaxis.set_major_formatter(mdates.DateFormatter("%Y"))

fig, ax = plt.subplots(figsize=(16,6), dpi=150)

ax.plot(tp_reg["time"].values, tp_reg.values, linewidth=1.3)

ax.set_title("Miesięczna suma opadu (średnia obszaru, 2005–2024)")
ax.set_xlabel("Rok")
ax.set_ylabel("Opad [mm/miesiąc]")
ax.grid(True)
ax.xaxis.set_major_locator(mdates.YearLocator(base=1))
ax.xaxis.set_minor_locator(mdates.MonthLocator())
ax.xaxis.set_major_formatter(mdates.DateFormatter("%Y"))

fig.tight_layout()
plt.show()
```

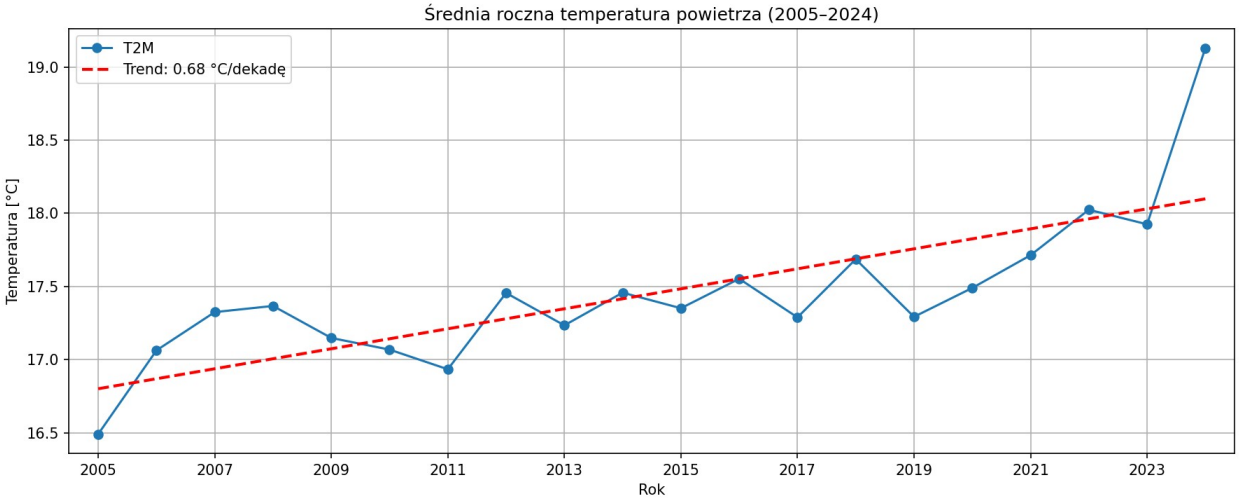




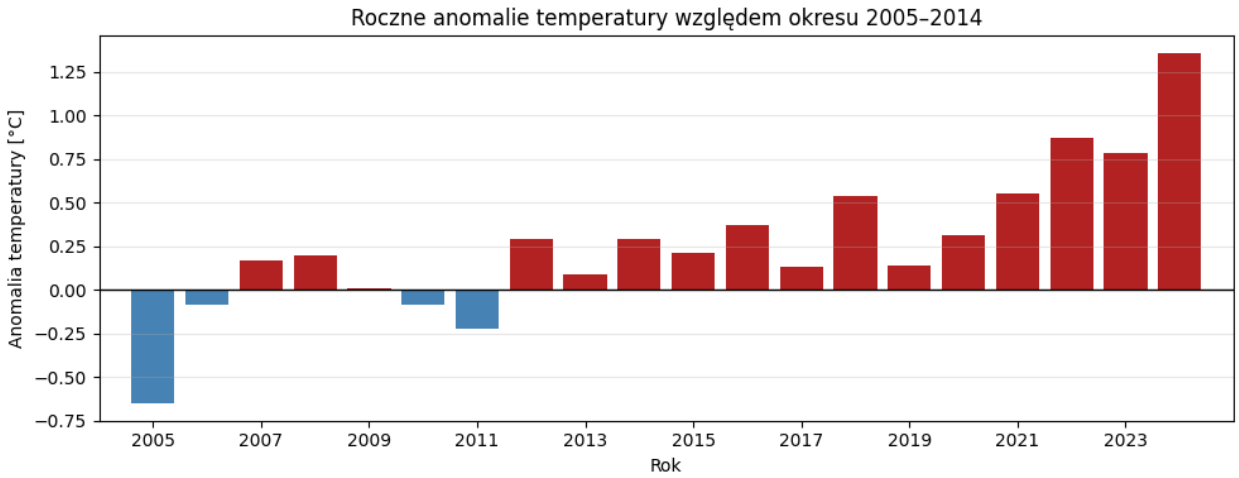

```
# Roczne: temperatura = średnia, opad = suma
t2m_y = t2m_reg.resample(time="YE").mean()
tp_y  = tp_reg.resample(time="YE").sum()

def trend_linreg(da_1d, label=""):
    """Trend liniowy dla 1D xarray: zwraca slope/rok oraz p-value."""
    y = da_1d.values.astype(float)
    x = da_1d["time"].dt.year.values.astype(float)
    mask = np.isfinite(x) & np.isfinite(y)
    slope, intercept, r, p, stderr = linregress(x[mask], y[mask])
    return slope, p, r

slope_t, p_t, r_t = trend_linreg(t2m_y, "t2m")
print(f"Trend T2M: {slope_t*10:.3f} °C/dekadę, p={p_t:.4f},
r={r_t:.3f}")

years = t2m_y["time"].dt.year.values.astype(int)
values = t2m_y.values.astype(float)

fig, ax = plt.subplots(figsize=(12,5), dpi=150)

ax.plot(years, values, marker="o", linewidth=1.5)
ax.set_title("Średnia roczna temperatura powietrza (2005–2024)")
ax.set_xlabel("Rok")
ax.set_ylabel("Temperatura [°C]")
ax.grid(True)
tick_years = years[::2]
ax.set_xticks(tick_years)
ax.set_xticklabels(tick_years, rotation=0)
ax.set_xlim(years.min() - 0.5, years.max() + 0.5)
fig.tight_layout()

plt.show()


Trend T2M: 0.683 °C/dekadę, p=0.0001, r=0.771
```



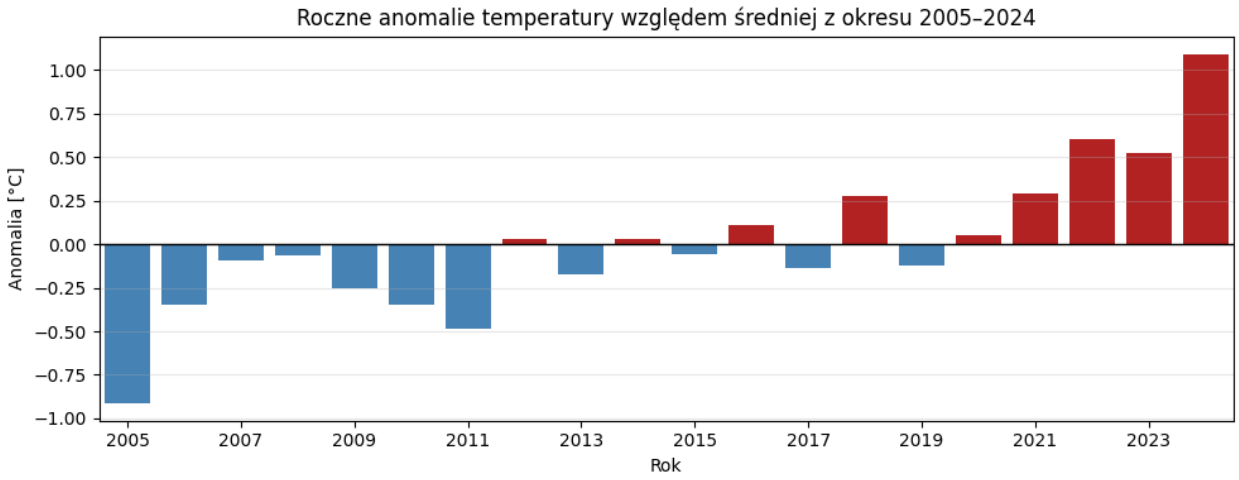

```python
def plot_trend(ax, years, values, slope, intercept, color="k",
label="Trend"):
    y_trend = slope * years + intercept
    ax.plot(years, y_trend, linestyle="--", linewidth=2,
            color=color, label=label)
# trend temperatura
slope_t, p_t, r_t = trend_linreg(t2m_y)
intercept_t = np.nanmean(values) - slope_t * np.nanmean(years)

fig, ax = plt.subplots(figsize=(12,5), dpi=150)

ax.plot(years, values, marker="o", linewidth=1.5, label="T2M")
plot_trend(
    ax, years, values,
    slope_t, intercept_t,
    color="red",
    label=f"Trend: {slope_t*10:.2f} °C/dekadę"
)

ax.set_title("Średnia roczna temperatura powietrza (2005–2024)")
ax.set_xlabel("Rok")
ax.set_ylabel("Temperatura [°C]")
ax.grid(True)
ax.legend()

ax.set_xticks(years[::2])
ax.set_xlim(years.min() - 0.5, years.max() + 0.5)

fig.tight_layout()
plt.show()
```



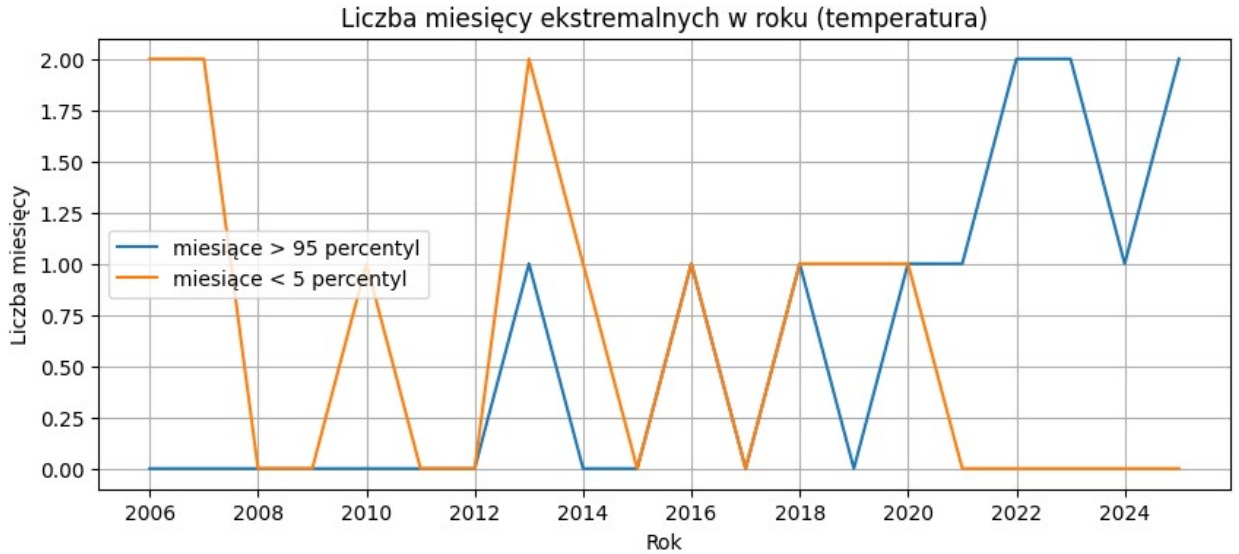

```
t2m_year = t2m_daily.resample(time="YS").mean()
ref_start = "2005-01-01"
ref_end   = "2014-12-31"

t2m_ref = t2m_year.sel(time=slice(ref_start, ref_end)).mean("time")
t2m_anom = t2m_year - t2m_ref
years = t2m_anom["time"].dt.year.values
values = t2m_anom.values

plt.figure(figsize=(10,4))
plt.bar(
    years,
    values,
    color=np.where(values >= 0, "firebrick", "steelblue"),
    width=0.8
)

plt.axhline(0, color="black", lw=1)

plt.title("Roczne anomalie temperatury względem okresu 2005–2014")
plt.xlabel("Rok")
plt.ylabel("Anomalia temperatury [°C]")

# skala osi X
plt.xticks(years[::2])   # co 2 lata (zmień na ::3 jeśli gęsto)
plt.xlim(years.min() - 1.0, years.max() + 1.0)

plt.grid(axis="y", alpha=0.3)
plt.tight_layout()
plt.show()
```




```
t2m_year = t2m_daily.resample(time="YE").mean()

ref_start = "2005-01-01"
ref_end   = "2024-12-31"

t2m_ref = t2m_year.sel(time=slice(ref_start, ref_end)).mean("time")
t2m_anom_0524 = t2m_year - t2m_ref

years  = t2m_anom_0524["time"].dt.year.values
values = t2m_anom_0524.values

plt.figure(figsize=(10,4))
plt.bar(years, values, color=np.where(values>=0, "firebrick",
"steelblue"), width=0.8)
plt.axhline(0, color="black", lw=1)

plt.title("Roczne anomalie temperatury względem średniej z okresu
2005–2024")
plt.xlabel("Rok"); plt.ylabel("Anomalia [°C]")

plt.xticks(years[::2])
plt.xlim(years.min()-0.5, years.max()+0.5)
plt.grid(axis="y", alpha=0.3)
plt.tight_layout()
plt.show()
```



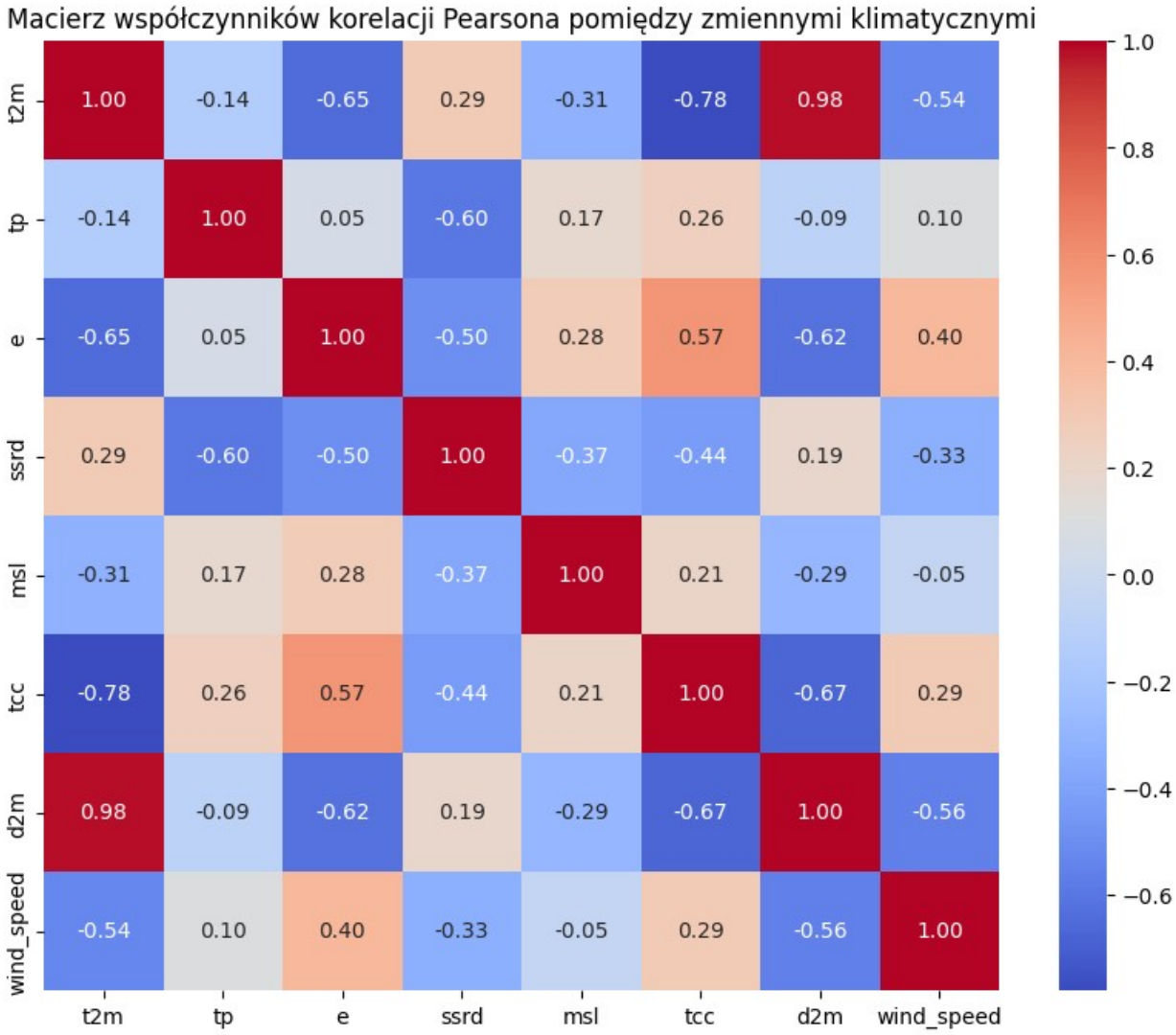

```
#ekstremalne miesieczne temperatury
hot_months = (t2m_reg >
t2m_reg.quantile(0.95)).resample(time="YE").sum()
cold_months = (t2m_reg <
t2m_reg.quantile(0.05)).resample(time="YE").sum()

plt.figure(figsize=(10,4))
plt.plot(hot_months["time"], hot_months, label="miesiące > 95
percentyl")
plt.plot(cold_months["time"], cold_months, label="miesiące < 5
percentyl")
plt.title("Liczba miesięcy ekstremalnych w roku (temperatura)")
plt.xlabel("Rok"); plt.ylabel("Liczba miesięcy"); plt.grid(True);
plt.legend()
plt.show()
```



```
vars_corr = ["t2m", "tp", "e", "ssrd", "msl", "tcc", "d2m",
"wind_speed"]

df = pd.DataFrame(
    {v: area_mean(ds_m[v], lat_slice, lon_slice).values for v in
vars_corr},
    index=pd.to_datetime(ds_m["time"].values)
)

corr = df.corr()
corr
```

```
                  t2m        tp         e      ssrd       msl       tcc
\
t2m          1.000000 -0.144899 -0.650125  0.289730 -0.313640 -0.778971

tp          -0.144899  1.000000  0.052155 -0.595064  0.173768  0.255769

e           -0.650125  0.052155  1.000000 -0.495615  0.281390  0.570914

ssrd         0.289730 -0.595064 -0.495615  1.000000 -0.374674 -0.438034

msl         -0.313640  0.173768  0.281390 -0.374674  1.000000  0.211911

tcc         -0.778971  0.255769  0.570914 -0.438034  0.211911  1.000000

d2m          0.981300 -0.090247 -0.620489  0.192559 -0.286229 -0.672606

wind_speed -0.535291  0.102527  0.403164 -0.333972 -0.048727  0.286380


                 d2m  wind_speed
t2m         0.981300   -0.535291
tp         -0.090247    0.102527
e          -0.620489    0.403164
ssrd        0.192559   -0.333972
msl        -0.286229   -0.048727
tcc        -0.672606    0.286380
d2m         1.000000   -0.557741
wind_speed -0.557741    1.000000
```

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(10,8))
sns.heatmap(corr, annot=True, fmt=".2f", cmap='coolwarm', cbar=True)
plt.title("Macierz współczynników korelacji Pearsona pomiędzy
zmiennymi klimatycznymi")
plt.show()
```

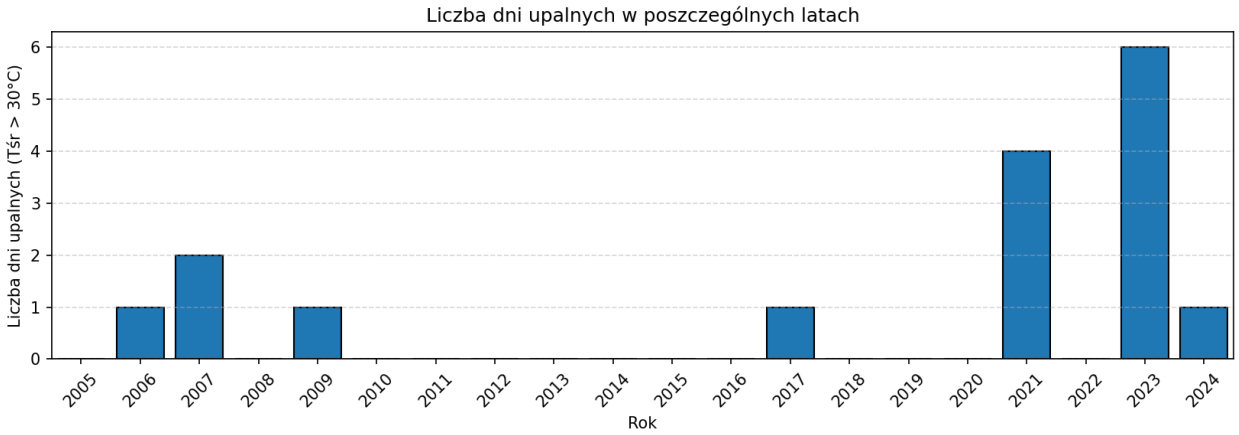



```
HOURLY_FILES = ["data1.grib", "data2.grib", "data3.grib",
"data4.grib"]

ds_t_list = []
for f in HOURLY_FILES:
    ds_t_list.append(xr.open_dataset(f, **CFGRIB_KW,
filter_by_keys={"shortName": "2t"}))

ds_temp = xr.concat(ds_t_list, dim="time").sortby("time")
ds_temp["t2m"] = ds_temp["t2m"] - 273.15

t2m_hour_reg = area_mean(ds_temp["t2m"], lat_slice, lon_slice)
t2m_daily = t2m_hour_reg.resample(time="1D").mean()

ds_temp.isnull().sum()
```

```
<xarray.Dataset> Size: 24B
Dimensions:  ()
Coordinates:
    number   int32 4B 0
    step     timedelta64[ns] 8B 00:00:00
    surface  float64 8B 0.0
Data variables:
    t2m      int32 4B 0
Attributes:
    GRIB_edition:            1
    GRIB_centre:             ecmf
    GRIB_centreDescription:  European Centre for Medium-Range Weather
Forecasts
    GRIB_subCentre:          0
    Conventions:             CF-1.7
    institution:             European Centre for Medium-Range Weather
Forecasts
    history:                 2026-01-05T11:02 GRIB to CDM+CF via
cfgrib-0.9.1...
```

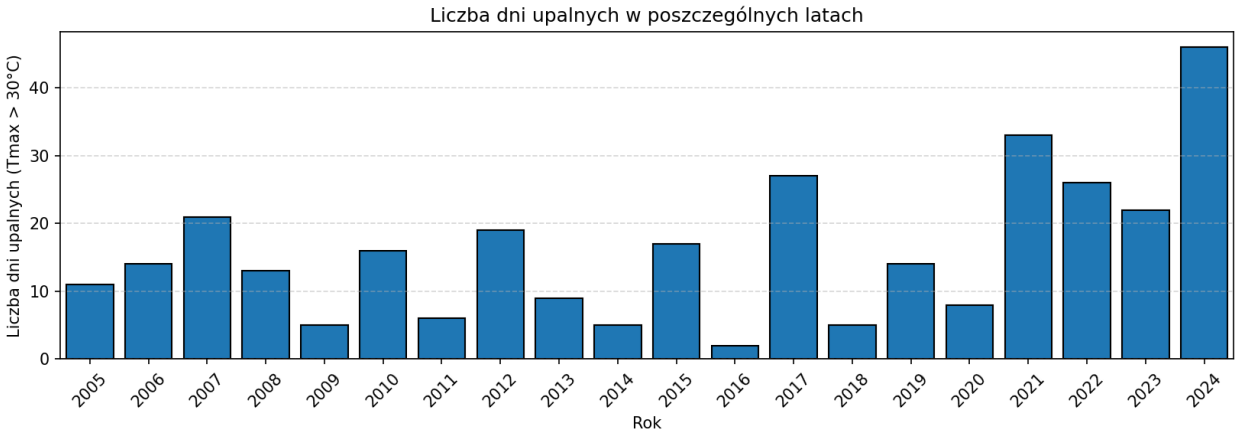
```python
t2m_daily_max= t2m_hour_reg.resample(time="1D").max()

hot_days_y = (t2m_daily > 30).resample(time="YS").sum()

slope_hd, p_hd, r_hd = trend_linreg(hot_days_y)
print(f"Trend dni upalnych: {slope_hd*10:.3f} dni/dekadę,
p={p_hd:.4f}, r={r_hd:.3f}")

years = hot_days_y["time"].dt.year.values.astype(int)
vals  = hot_days_y.values.astype(int)

plt.figure(figsize=(11,4), dpi=150)
plt.bar(years, vals, width=0.8, edgecolor="black")

plt.xlabel("Rok")
plt.ylabel("Liczba dni upalnych (Tśr > 30°C)")
plt.title("Liczba dni upalnych w poszczególnych latach")
plt.grid(axis="y", linestyle="--", alpha=0.5)

plt.xticks(years, rotation=45)
plt.xlim(years.min() - 0.5, years.max() + 0.5)

plt.tight_layout()
plt.show()
#nieistotne statystycznie, nie wykorzystane w raporcie
```

```
Trend dni upalnych: 0.902 dni/dekadę, p=0.1441, r=0.339
```

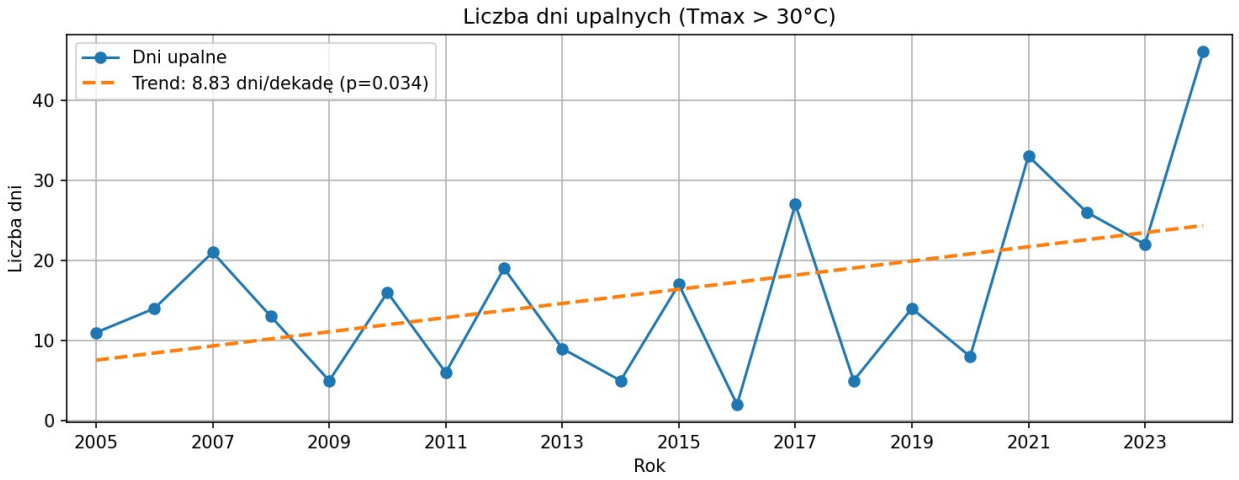



```
hot_days_y = (t2m_daily_max > 30).resample(time="YS").sum()

slope_hd, p_hd, r_hd = trend_linreg(hot_days_y)
print(f"Trend dni upalnych: {slope_hd*10:.3f} dni/dekadę,
p={p_hd:.4f}, r={r_hd:.3f}")

years = hot_days_y["time"].dt.year.values.astype(int)
vals  = hot_days_y.values.astype(int)

plt.figure(figsize=(11,4), dpi=150)
plt.bar(years, vals, width=0.8, edgecolor="black")

plt.xlabel("Rok")
plt.ylabel("Liczba dni upalnych (Tmax > 30°C)")
plt.title("Liczba dni upalnych w poszczególnych latach")
plt.grid(axis="y", linestyle="--", alpha=0.5)

plt.xticks(years, rotation=45)
plt.xlim(years.min() - 0.5, years.max() + 0.5)

plt.tight_layout()
plt.show()

#istotne, wykorzytsane w raporcie
```

```
Trend dni upalnych: 8.835 dni/dekadę, p=0.0341, r=0.476
```

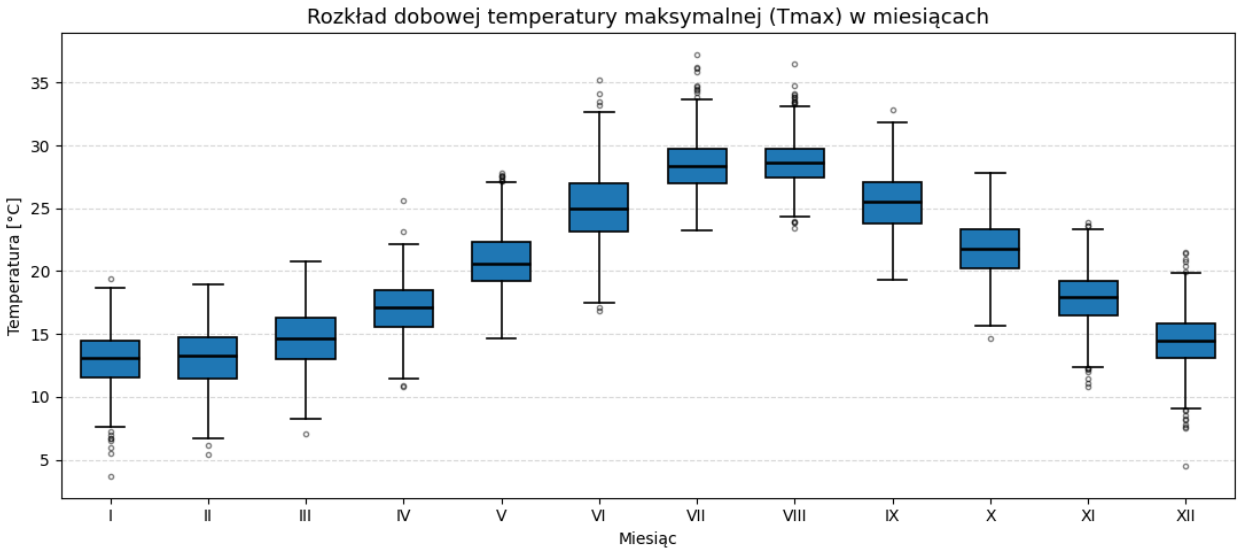



```python
import numpy as np
import matplotlib.pyplot as plt

# dane
years = hot_days_y["time"].dt.year.values.astype(int)
values = hot_days_y.values.astype(float)

# trend (już masz slope_hd w dniach/rok)
intercept_hd = np.nanmean(values) - slope_hd * np.nanmean(years)
trend = intercept_hd + slope_hd * years

# wykres
fig, ax = plt.subplots(figsize=(10,4), dpi=150)

ax.plot(years, values, marker="o", linewidth=1.5, label="Dni upalne")
ax.plot(
    years, trend,
    linestyle="--", linewidth=2,
    label=f"Trend: {slope_hd*10:.2f} dni/dekadę (p={p_hd:.3f})"
)

ax.set_title("Liczba dni upalnych (Tmax > 30°C)")
ax.set_xlabel("Rok")
ax.set_ylabel("Liczba dni")
ax.grid(True)

ax.set_xticks(years[::2])
ax.set_xlim(years.min() - 0.5, years.max() + 0.5)
ax.legend()

fig.tight_layout()
plt.show()
```

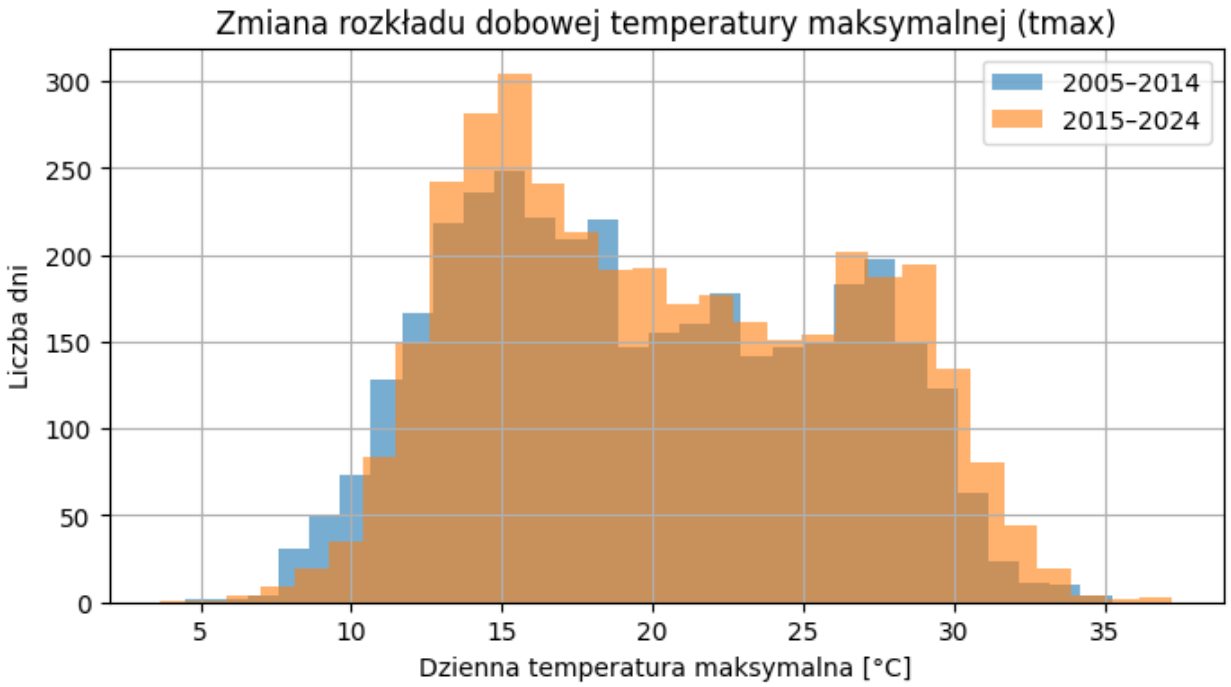



```
import matplotlib.pyplot as plt
import numpy as np

df = t2m_daily_max.to_dataframe(name="Tmax")
df["month"] = df.index.month
data = [df[df["month"] == m]["Tmax"].dropna() for m in range(1, 13)]

fig, ax = plt.subplots(figsize=(11, 5))

bp = ax.boxplot(
    data,
    patch_artist=True,
    widths=0.6,
    showfliers=True,
    medianprops=dict(color="black", linewidth=1.8),
    boxprops=dict(linewidth=1.2),
    whiskerprops=dict(linewidth=1.2),
    capprops=dict(linewidth=1.2),
    flierprops=dict(marker="o", markersize=3, alpha=0.5)
)

ax.set_xticks(range(1, 13))
ax.set_xticklabels(
    ["I","II","III","IV","V","VI","VII","VIII","IX","X","XI","XII"]
)

ax.set_title("Rozkład dobowej temperatury maksymalnej (Tmax) w
miesiącach", fontsize=13)
ax.set_xlabel("Miesiąc")
ax.set_ylabel("Temperatura [°C]")

ax.grid(True, axis="y", linestyle="--", alpha=0.5)
ax.set_axisbelow(True)
```

```
plt.tight_layout()
plt.show()
```

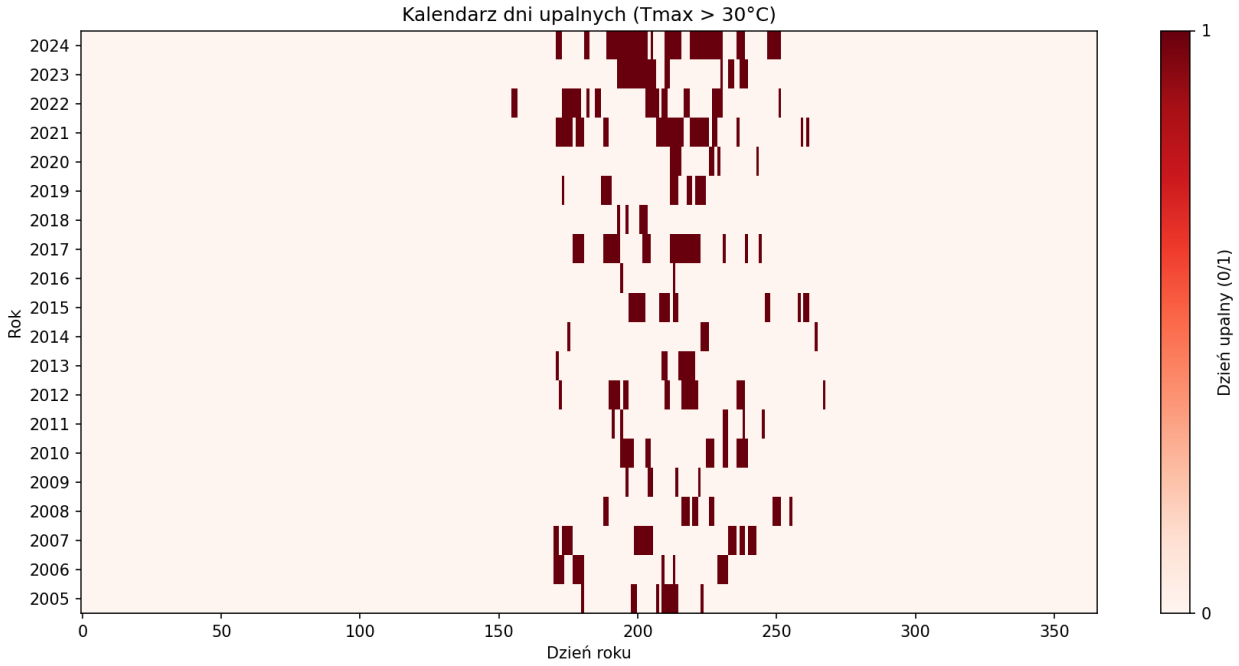



```
t1 = t2m_daily_max.sel(time=slice("2005","2014"))
t2 = t2m_daily_max.sel(time=slice("2015","2024"))

plt.figure(figsize=(8,4))
plt.hist(t1, bins=30, alpha=0.6, label="2005–2014")
plt.hist(t2, bins=30, alpha=0.6, label="2015–2024")
plt.xlabel("Dzienna temperatura maksymalna [°C]"); plt.ylabel("Liczba
dni")
plt.legend(); plt.grid(True)
plt.title("Zmiana rozkładu dobowej temperatury maksymalnej (tmax)")
plt.show()
```




```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# 1) maska dni upalnych jako 0/1 (int) – kluczowe!
hot = (t2m_daily_max > 30).astype(np.int8)

# 2) do DataFrame
df = hot.to_dataframe(name="hot").reset_index()

# 3) rok i dzień roku (kolumna 'time' po reset_index)
df["year"] = pd.to_datetime(df["time"]).dt.year
df["doy"]  = pd.to_datetime(df["time"]).dt.dayofyear

# 4) pivot (rok × dzień roku)
pivot = df.pivot_table(
    index="year",
    columns="doy",
    values="hot",
    aggfunc="max",
    fill_value=0
).astype(np.int8)  # wymuszamy liczbowy typ

# 5) rysowanie
fig, ax = plt.subplots(figsize=(12, 6), dpi=150)

im = ax.imshow(
    pivot.to_numpy(),
```

```
    aspect="auto",
    cmap="Reds",
    origin="lower",
    vmin=0,
    vmax=1
)

years = pivot.index.to_numpy()
ax.set_yticks(np.arange(len(years)))
ax.set_yticklabels(years)

ax.set_xlabel("Dzień roku")
ax.set_ylabel("Rok")
ax.set_title("Kalendarz dni upalnych (Tmax > 30°C)")

cbar = fig.colorbar(im, ax=ax)
cbar.set_label("Dzień upalny (0/1)")
cbar.set_ticks([0, 1])

plt.tight_layout()
plt.show()
```




```
#min lokalne
tmin_abs=ds_temp["t2m"].min()
tmin_loc = ds_temp["t2m"].where(
    ds_temp["t2m"] == tmin_abs,
    drop=True
)
```

```
tmin_loc
```

```
<xarray.DataArray 't2m' (time: 1, latitude: 1, longitude: 1)> Size: 4B
array([[[-6.517914]]], dtype=float32)
Coordinates:
  * time        (time) datetime64[ns] 8B 2019-01-05T01:00:00
  * latitude    (latitude) float64 8B 37.8
  * longitude   (longitude) float64 8B 14.9
    number      int32 4B 0
    step        timedelta64[ns] 8B 00:00:00
    surface     float64 8B 0.0
    valid_time  (time) datetime64[ns] 8B 2019-01-05T01:00:00
Attributes: (12/31)
    GRIB_paramId:                    167
    GRIB_dataType:                   an
    GRIB_numberOfPoints:             9
    GRIB_typeOfLevel:                surface
    GRIB_stepUnits:                  1
    GRIB_stepType:                   instant
    ...                              ...
    GRIB_shortName:                  2t
    GRIB_totalNumber:                0
    GRIB_units:                      K
    long_name:                       2 metre temperature
    units:                           K
    standard_name:                   unknown
```

```
#max lokalne
tmax_abs = ds_temp["t2m"].max()
tmax_loc = ds_temp["t2m"].where(
    ds_temp["t2m"] == tmax_abs,
    drop=True
)

tmax_loc
```

```
<xarray.DataArray 't2m' (time: 1, latitude: 1, longitude: 1)> Size: 4B
array([[[42.339996]]], dtype=float32)
Coordinates:
  * time        (time) datetime64[ns] 8B 2009-07-25T13:00:00
  * latitude    (latitude) float64 8B 37.3
  * longitude   (longitude) float64 8B 14.9
    number      int32 4B 0
    step        timedelta64[ns] 8B 00:00:00
    surface     float64 8B 0.0
    valid_time  (time) datetime64[ns] 8B 2009-07-25T13:00:00
Attributes: (12/31)
    GRIB_paramId:                    167
    GRIB_dataType:                   an
```

```
    GRIB_numberOfPoints:                    9
    GRIB_typeOfLevel:                       surface
    GRIB_stepUnits:                         1
    GRIB_stepType:                          instant
    ...                                     ...
    GRIB_shortName:                         2t
    GRIB_totalNumber:                       0
    GRIB_units:                             K
    long_name:                              2 metre temperature
    units:                                  K
    standard_name:                          unknown
```

```
#min dla sr obszaru
t2m_hour_reg = area_mean(ds_temp["t2m"], lat_slice, lon_slice)
tmin_reg = t2m_hour_reg.min()

tmin_reg
```

```
<xarray.DataArray 't2m' ()> Size: 8B
array(0.68192236)
Coordinates:
    number   int32 4B 0
    step     timedelta64[ns] 8B 00:00:00
    surface  float64 8B 0.0
Attributes: (12/31)
    GRIB_paramId:                           167
    GRIB_dataType:                          an
    GRIB_numberOfPoints:                    9
    GRIB_typeOfLevel:                       surface
    GRIB_stepUnits:                         1
    GRIB_stepType:                          instant
    ...                                     ...
    GRIB_shortName:                         2t
    GRIB_totalNumber:                       0
    GRIB_units:                             K
    long_name:                              2 metre temperature
    units:                                  K
    standard_name:                          unknown
```

```
#max dla sr obszaru
tmax_reg = t2m_hour_reg.max()

tmax_reg
```

```
<xarray.DataArray 't2m' ()> Size: 8B
array(37.26040671)
Coordinates:
    number   int32 4B 0
    step     timedelta64[ns] 8B 00:00:00
```

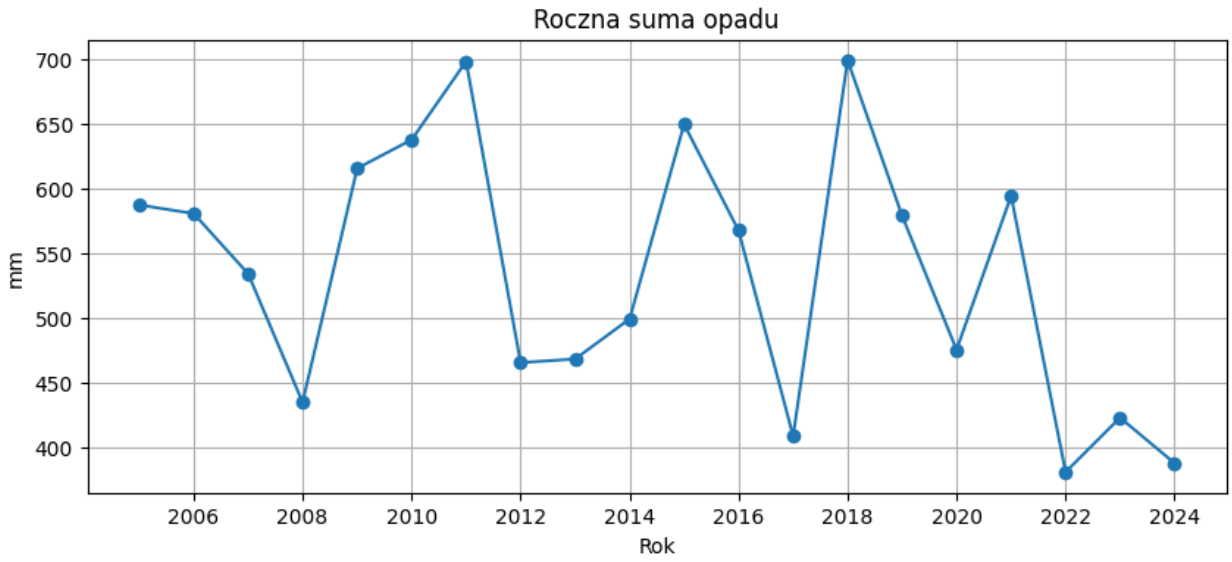
```
    surface  float64 8B 0.0
Attributes: (12/31)
    GRIB_paramId:                            167
    GRIB_dataType:                           an
    GRIB_numberOfPoints:                     9
    GRIB_typeOfLevel:                        surface
    GRIB_stepUnits:                          1
    GRIB_stepType:                           instant
    ...                                      ...
    GRIB_shortName:                          2t
    GRIB_totalNumber:                        0
    GRIB_units:                              K
    long_name:                               2 metre temperature
    units:                                   K
    standard_name:                           unknown
```

```
ds_p_list = []
for f in HOURLY_FILES:
    ds_p_list.append(
        xr.open_dataset(
            f,
            **CFGRIB_KW,
            filter_by_keys={"shortName": "tp", "typeOfLevel":
"surface", "stepType": "accum"}
        )
    )

ds_opad = xr.concat(ds_p_list, dim="time").sortby("time")

tp = ds_opad["tp"] * 1000.0

tp_inc = tp.diff("time")
tp_inc = tp_inc.where(tp_inc >= 0)



tp_hour_reg = tp_inc.resample(time="1D").sum()
tp_daily = area_mean(tp_hour_reg, lat_slice, lon_slice)
# próg ekstremalnego opadu: np. 99.9 percentyl dni mokrych

tp_daily.isnull().sum()
```

```
<xarray.DataArray 'tp' ()> Size: 4B
array(0)
Coordinates:
    number   int32 4B 0
    surface  float64 8B 0.0
Attributes: (12/31)
    GRIB_paramId:                            228
    GRIB_dataType:                           fc
```

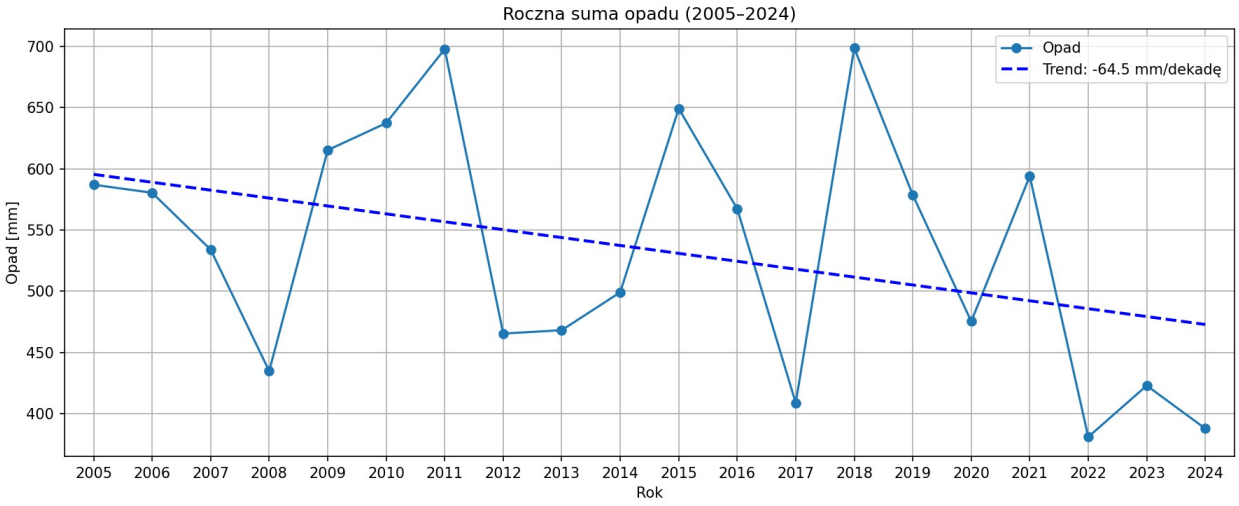
```
    GRIB_numberOfPoints:                    9
    GRIB_typeOfLevel:                       surface
    GRIB_stepUnits:                         1
    GRIB_stepType:                          accum
    ...                                     ...
    GRIB_shortName:                         tp
    GRIB_totalNumber:                       0
    GRIB_units:                             m
    long_name:                              Total precipitation
    units:                                  m
    standard_name:                          unknown
```

```
tp_daily_1d = tp_daily.sum(dim="step")
tp_year = tp_daily_1d.resample(time="YS").sum()


plt.figure(figsize=(10,4))
plt.plot(tp_year["time"], tp_year, marker="o")
plt.title("Roczna suma opadu")
plt.xlabel("Rok"); plt.ylabel("mm")
plt.grid(True)
plt.show()
```




```
# dane opadu
years1 = tp_year["time"].dt.year.values.astype(int)
values1 = tp_year.values.astype(float)

# trend opadu
slope_p, p_p, r_p = trend_linreg(tp_year)
intercept_p = np.nanmean(values1) - slope_p * np.nanmean(years1)

fig, ax = plt.subplots(figsize=(12,5), dpi=150)
```

```
ax.plot(years1, values1, marker="o", linewidth=1.5, label="Opad")
plot_trend(
    ax, years1, values1,
    slope_p, intercept_p,
    color="blue",
    label=f"Trend: {slope_p*10:.1f} mm/dekadę"
)

ax.set_title("Roczna suma opadu (2005–2024)")
ax.set_xlabel("Rok")
ax.set_ylabel("Opad [mm]")
ax.grid(True)
ax.legend()

ax.set_xticks(years1)
ax.set_xlim(years1.min() - 0.5, years1.max() + 0.5)

fig.tight_layout()
plt.show()
```

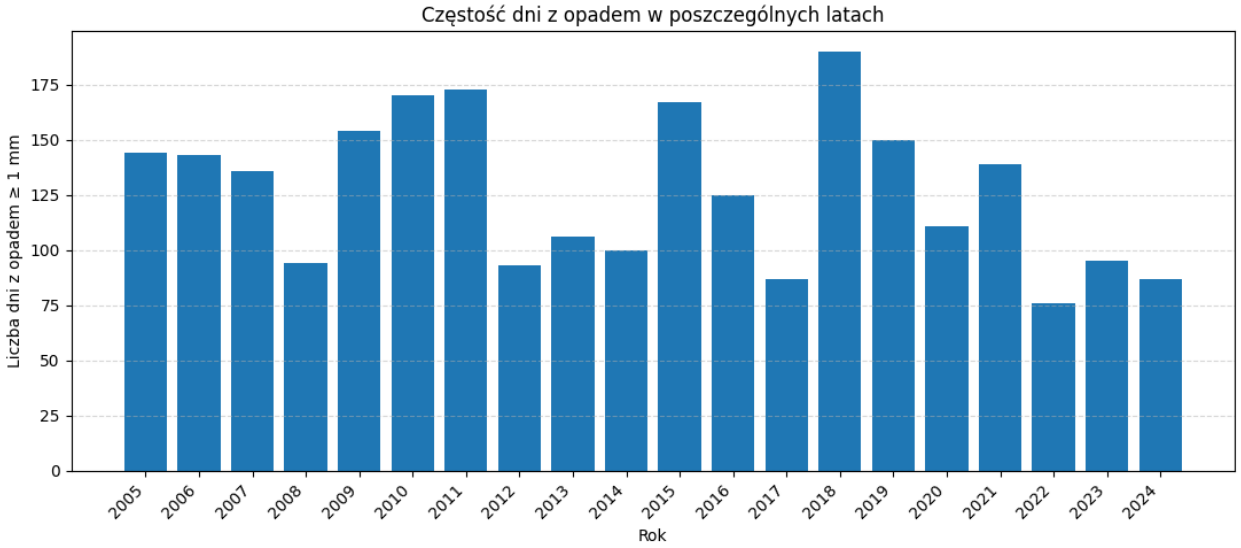



```
from scipy import stats
import numpy as np

x = years1
y = values1

mask = np.isfinite(x) & np.isfinite(y)
x = x[mask]
y = y[mask]

res = stats.linregress(x, y)

slope = res.slope
```

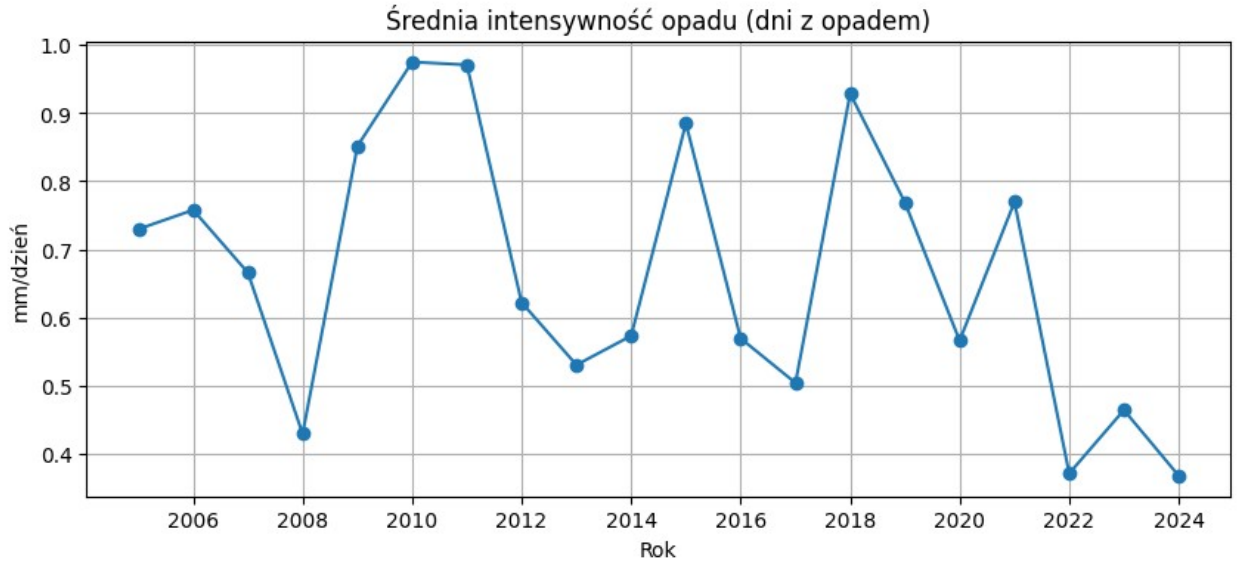
```
intercept = res.intercept
p_value = res.pvalue
r_value = res.rvalue

print(f"Slope: {slope*10:.1f} mm/dekadę")
print(f"p-value: {p_value:.4f}")
print(f"R: {r_value:.3f}")
```

```
Slope: -64.5 mm/dekadę
p-value: 0.0991
R: -0.379
```

```
wet_days = (tp_daily >= 1.0)

wet_days_1d = wet_days.sum(dim="step") if "step" in wet_days.dims else
wet_days

wet_days_year = wet_days_1d.resample(time="YE").sum()

years  = wet_days_year["time"].dt.year.values
values = wet_days_year.values

fig, ax = plt.subplots(figsize=(11,5))
ax.bar(years, values, width=0.8)

ax.set_xlabel("Rok")
ax.set_ylabel("Liczba dni z opadem ≥ 1 mm")
ax.set_title("Częstość dni z opadem w poszczególnych latach")

ax.set_xticks(years)                # pokaż każdy rok
ax.set_xticklabels(years, rotation=45, ha="right")

ax.grid(axis="y", linestyle="--", alpha=0.5)
plt.tight_layout()
plt.show()
```

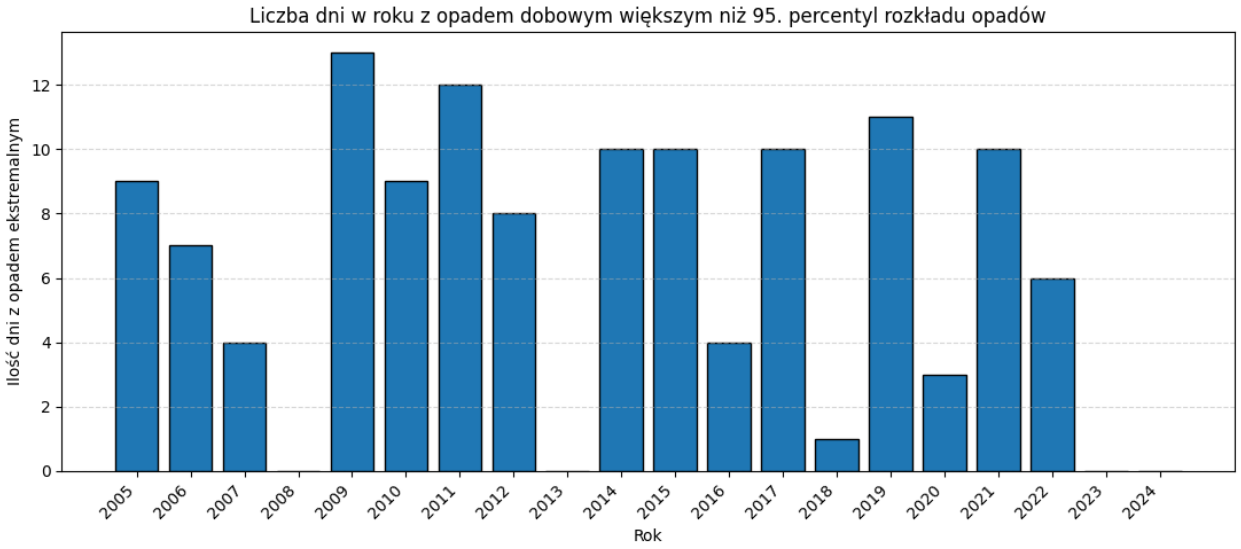



```
intensity = tp_daily.where(tp_daily >= 1.0)
intensity_1d=intensity.sum(dim="step")
intensity_year = intensity_1d.resample(time="YS").mean()

plt.figure(figsize=(10,4))
plt.plot(intensity_year["time"], intensity_year, marker="o")
plt.title("Średnia intensywność opadu (dni z opadem)")
plt.xlabel("Rok"); plt.ylabel("mm/dzień")
plt.grid(True)
plt.show()
```



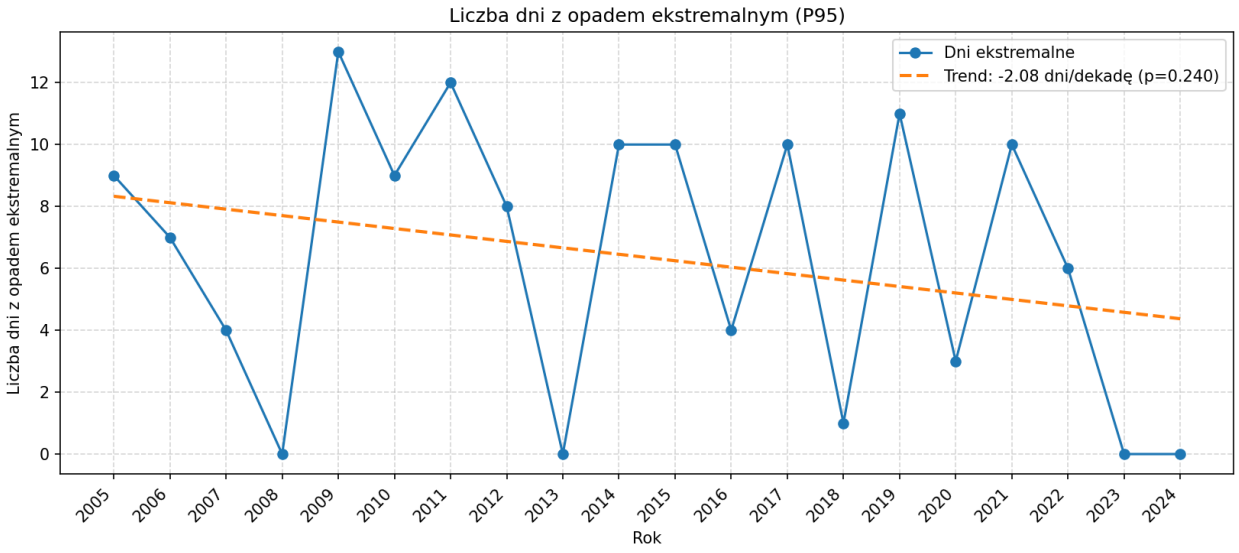

```
import matplotlib.pyplot as plt
wet = tp_daily.where(tp_daily >= 1.0)
p95_wet = wet.quantile(0.95, skipna=True)
```

```
extreme_rain = tp_daily >= p95_wet

extreme_rain_1d = extreme_rain.sum(dim="step") if "step" in
extreme_rain.dims else extreme_rain

extreme_rain_year = extreme_rain_1d.resample(time="YE").sum()

years  = extreme_rain_year["time"].dt.year.values
values = extreme_rain_year.values

fig, ax = plt.subplots(figsize=(11,5))
ax.bar(years, values, width=0.8,edgecolor="black")

ax.set_xlabel("Rok")
ax.set_ylabel("Ilość dni z opadem ekstremalnym")
ax.set_title("Liczba dni w roku z opadem dobowym większym niż 95.
percentyl rozkładu opadów")

ax.set_xticks(years)                # pokaż każdy rok
ax.set_xticklabels(years, rotation=45, ha="right")

ax.grid(axis="y", linestyle="--", alpha=0.5)
plt.tight_layout()
plt.show()
```




```
mask = np.isfinite(years) & np.isfinite(values)
slope, intercept, r, p, stderr = linregress(years[mask], values[mask])

trend = intercept + slope * years

fig, ax = plt.subplots(figsize=(11,5), dpi=150)
```

```
ax.plot(years, values, marker="o", linewidth=1.5, label="Dni
ekstremalne")
ax.plot(
    years, trend,
    linestyle="--",
    linewidth=2,
    label=f"Trend: {slope*10:.2f} dni/dekadę (p={p:.3f})"
)

ax.set_xlabel("Rok")
ax.set_ylabel("Liczba dni z opadem ekstremalnym")
ax.set_title("Liczba dni z opadem ekstremalnym (P95)")

ax.set_xticks(years)
ax.set_xticklabels(years.astype(int), rotation=45, ha="right")

ax.grid(True, linestyle="--", alpha=0.5)
ax.legend()

plt.tight_layout()
plt.show()
#p-value nie jest istotne statystycznie
```




```
wet_frac = (tp_daily > 0).mean()
print("Udział dni z opadem >0:", float(wet_frac.values) if
wet_frac.size == 1 else wet_frac)
```

```
Udział dni z opadem >0: 0.6925279488934519
```

```
extreme_frac = extreme_rain.mean()
print("Udział dni oznaczonych jako >P95:", float(extreme_frac.values)
if extreme_frac.size == 1 else extreme_frac)
```

```
Udział dni oznaczonych jako >P95: 0.0014487793748574037
```

```
# bierzemy tylko godziny z opadem
tp_pos = tp_hour_reg.where(tp_hour_reg > 0, drop=True)

tp_min = float(tp_pos.min())
tp_max = float(tp_pos.max())

q25 = float(tp_pos.quantile(0.25))
q50 = float(tp_pos.quantile(0.50))
q75 = float(tp_pos.quantile(0.75))
q95 = float(tp_pos.quantile(0.95))

print("Statystyki godzinowych opadów [mm/h] (tylko godziny z
opadem):")
print(f"Min : {tp_min:.2f}")
print(f"Q25 : {q25:.2f}")
print(f"Q50 : {q50:.2f}")
print(f"Q75 : {q75:.2f}")
print(f"Q95 : {q95:.2f}")
print(f"Max : {tp_max:.2f}")
```

```
Statystyki godzinowych opadów [mm/h] (tylko godziny z opadem):
Min : 0.00
Q25 : 0.00
Q50 : 0.03
Q75 : 0.18
Q95 : 1.25
Max : 13.94
```

```
tp_month = tp_daily.resample(time="MS").sum()
import pandas as pd
import matplotlib.pyplot as plt

df = tp_month.to_dataframe(name="tp").dropna()
df["month"] = df.index.month

data = [df[df["month"] == m]["tp"].values for m in range(1, 13)]

fig, ax = plt.subplots(figsize=(11,5), dpi=150)

bp = ax.boxplot(
    data,
    patch_artist=True,
    widths=0.6,
    showfliers=True,
```

```
    medianprops=dict(color="black", linewidth=1.8),
    boxprops=dict(linewidth=1.2),
    whiskerprops=dict(linewidth=1.2),
    capprops=dict(linewidth=1.2),
    flierprops=dict(marker="o", markersize=3, alpha=0.4)
)

ax.set_xticks(range(1, 13))
ax.set_xticklabels(
    ["I","II","III","IV","V","VI","VII","VIII","IX","X","XI","XII"]
)

ax.set_title("Rozkład miesięcznych sum opadu (2005–2024)",
fontsize=13)
ax.set_xlabel("Miesiąc")
ax.set_ylabel("Suma opadu [mm/miesiąc]")

ax.grid(True, axis="y", linestyle="--", alpha=0.5)
ax.set_axisbelow(True)

plt.tight_layout()
plt.show()
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[43], line 6
      3 import matplotlib.pyplot as plt
      5 df = tp_month.to_dataframe(name="tp").dropna()
----> 6 df["month"] = df.index.month
      8 data = [df[df["month"] == m]["tp"].values for m in range(1, 13)]
     10 fig, ax = plt.subplots(figsize=(11,5), dpi=150)

AttributeError: 'MultiIndex' object has no attribute 'month'
```

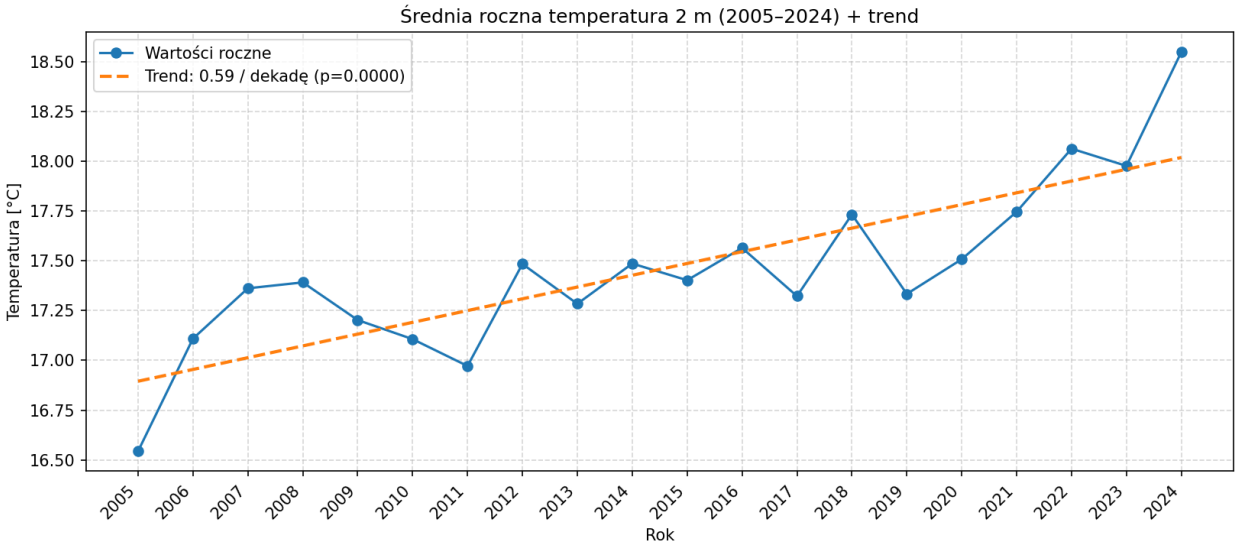
```
print(tp_hour_reg.max().item())
for q in [0.99, 0.995, 0.999]:
    print(q, tp_hour_reg.quantile(q).item())
(tp_hour_reg >= 50).sum().item()
```

```
13.93913459777832
0.99 2.2347296524047833
0.995 3.061963949203533
0.999 5.129454419612866

0
```

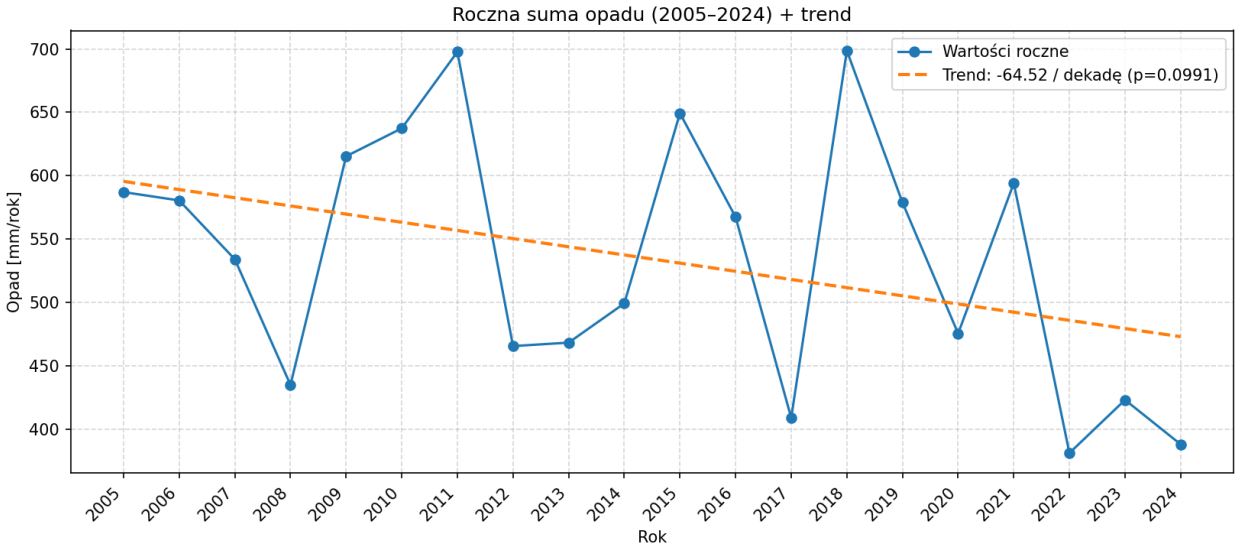
```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import linregress

def annualize_mean(da):
    """Roczne średnie (dla temp. itd.)"""
    return da.resample(time="YE").mean()

def annualize_sum(da):
    """Roczne sumy (dla opadu itd.)"""
    return da.resample(time="YE").sum()

def seasonal_mean(da):
    """Średnie sezonowe DJF/MAM/JJA/SON"""
    # xarray: groupby('time.season') działa poprawnie dla serii
dziennej/godzinowej
    return da.groupby("time.season").resample(time="YE").mean()

def seasonal_sum(da):
    """Sumy sezonowe DJF/MAM/JJA/SON (np. opad)"""
    return da.groupby("time.season").resample(time="YE").sum()

def plot_trend_yearly(year_da, title, ylabel):
    """Wykres liniowy + trend (na dekadę) dla rocznych wartości."""
    years = year_da["time"].dt.year.values.astype(float)
    vals = year_da.values.astype(float)

    m = np.isfinite(years) & np.isfinite(vals)
    slope, intercept, r, p, stderr = linregress(years[m], vals[m])
    trend = intercept + slope * years

    fig, ax = plt.subplots(figsize=(11,5), dpi=150)
    ax.plot(years, vals, marker="o", linewidth=1.5, label="Wartości
roczne")
    ax.plot(years, trend, linestyle="--", linewidth=2,
            label=f"Trend: {slope*10:.2f} / dekadę (p={p:.4f})")
    ax.set_title(title)
    ax.set_xlabel("Rok")
    ax.set_ylabel(ylabel)
    ax.set_xticks(years)
    ax.set_xticklabels(years.astype(int), rotation=45, ha="right")
    ax.grid(True, linestyle="--", alpha=0.5)
    ax.legend()
    plt.tight_layout()
    plt.show()

def plot_trend_seasonal(season_year_da, title, ylabel):
    """
    season_year_da: DataArray z wymiarem season i time (po
```

```
groupby('time.season').resample('YE'))
    Rysuje 4 sezonowe serie + trendy w osobnych panelach.
    """
    seasons = ["DJF", "MAM", "JJA", "SON"]
    fig, axes = plt.subplots(2, 2, figsize=(12,7), dpi=150,
sharex=True)
    axes = axes.ravel()

    for i, s in enumerate(seasons):
        ax = axes[i]
        da_s = season_year_da.sel(season=s)
        years = da_s["time"].dt.year.values.astype(float)
        vals = da_s.values.astype(float)

        m = np.isfinite(years) & np.isfinite(vals)
        slope, intercept, r, p, stderr = linregress(years[m], vals[m])
        trend = intercept + slope * years

        ax.plot(years, vals, marker="o", linewidth=1.2)
        ax.plot(years, trend, linestyle="--", linewidth=2)
        ax.set_title(f"{s}: {slope*10:.2f}/dekadę, p={p:.3f}")
        ax.grid(True, linestyle="--", alpha=0.4)

    for ax in axes[-2:]:
        ax.set_xlabel("Rok")
    for ax in axes[::2]:
        ax.set_ylabel(ylabel)

    fig.suptitle(title, y=1.02, fontsize=13)
    plt.tight_layout()
    plt.show()
def seasonal_series_sum(da_1d):
    """
    da_1d: DataArray o wymiarach ('time',)
    Zwraca DataArray o wymiarach ('season','time') z rocznymi sumami
dla DJF/MAM/JJA/SON.
    """
    q = da_1d.resample(time="QS-DEC").sum()   # sezony: DJF, MAM, JJA,
SON
    season = q["time"].dt.season

    out = []
    for s in ["DJF", "MAM", "JJA", "SON"]:
        out.append(
            q.where(season == s, drop=True).resample(time="YE").sum()
        )

    return xr.concat(out,
dim="season").assign_coords(season=["DJF","MAM","JJA","SON"])
```

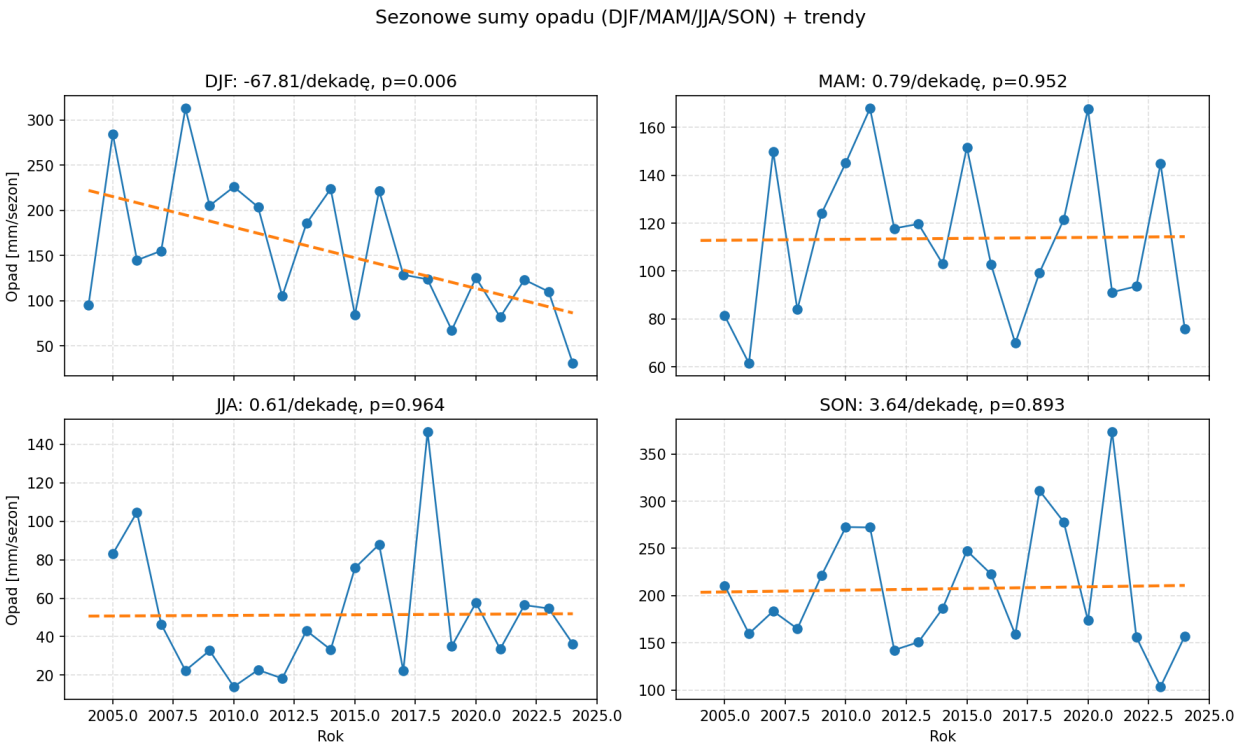
```
#trend roczny temperatury
t2m_year = annualize_mean(t2m_daily)
plot_trend_yearly(
    t2m_year,
    title="Średnia roczna temperatura 2 m (2005–2024) + trend",
    ylabel="Temperatura [°C]"
)
```



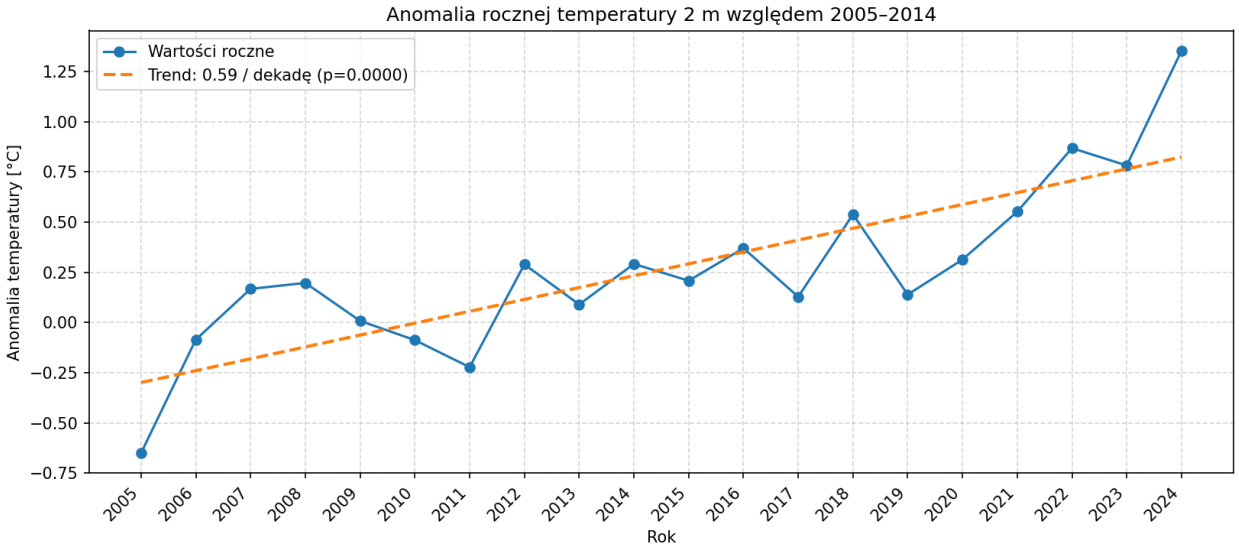

```
#trend roczny opadu
tp_daily=tp_daily.sum(dim="step")
tp_year = annualize_sum(tp_daily)
plot_trend_yearly(
    tp_year,
    title="Roczna suma opadu (2005–2024) + trend",
    ylabel="Opad [mm/rok]"
)
```




```
#trend sezonowy opadu

tp_season_year = seasonal_series_sum(tp_daily)
plot_trend_seasonal(
    tp_season_year,
    title="Sezonowe sumy opadu (DJF/MAM/JJA/SON) + trendy",
    ylabel="Opad [mm/sezon]"
)


C:\Users\kajas\AppData\Local\Temp\ipykernel_7256\658334948.py:92:
FutureWarning: In a future version of xarray the default value for
join will change from join='outer' to join='exact'. This change will
result in the following ValueError: cannot be aligned with
join='exact' because index/labels/sizes are not equal along these
coordinates (dimensions): 'time' ('time',) The recommendation is to
set join explicitly for this case.
  return xr.concat(out,
dim="season").assign_coords(season=["DJF","MAM","JJA","SON"])
```

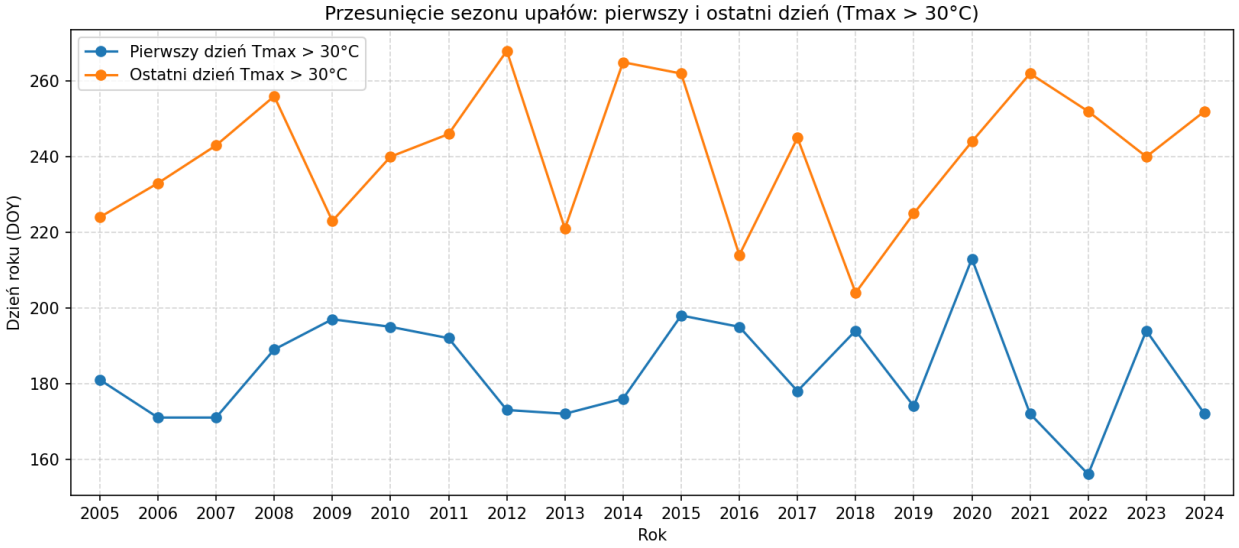



```
#anomalie temperatury rocznej wzgledem 2005-2014
ref_start, ref_end = "2005-01-01", "2014-12-31"

t2m_year = annualize_mean(t2m_daily)
ref_mean = t2m_year.sel(time=slice(ref_start, ref_end)).mean("time")

t2m_anom = t2m_year - ref_mean

plot_trend_yearly(
    t2m_anom,
    title="Anomalia rocznej temperatury 2 m względem 2005–2014",
    ylabel="Anomalia temperatury [°C]"
)
```




```
#przesuniecia sezonow (sezon upalow)
hot = (t2m_daily_max > 30)

df = hot.to_dataframe(name="hot").reset_index()
df["year"] = pd.to_datetime(df["time"]).dt.year
df["doy"]  = pd.to_datetime(df["time"]).dt.dayofyear

# wyciągamy pierwszy i ostatni dzień upału w roku
first_hot = df[df["hot"]].groupby("year")["doy"].min()
last_hot  = df[df["hot"]].groupby("year")["doy"].max()

season_len = (last_hot - first_hot + 1).dropna()

# wykres: początek i koniec sezonu
years = first_hot.index.values.astype(int)

fig, ax = plt.subplots(figsize=(11,5), dpi=150)
ax.plot(years, first_hot.values, marker="o", label="Pierwszy dzień
Tmax > 30°C")
ax.plot(years, last_hot.values, marker="o", label="Ostatni dzień Tmax
> 30°C")
ax.set_xlabel("Rok")
ax.set_ylabel("Dzień roku (DOY)")
ax.set_title("Przesunięcie sezonu upałów: pierwszy i ostatni dzień
(Tmax > 30°C)")
ax.grid(True, linestyle="--", alpha=0.5)
ax.legend()

ax.set_xticks(years)
ax.set_xlim(years.min() - 0.5, years.max() + 0.5)
plt.tight_layout()
plt.show()
```

```
# wykres: długość sezonu upałów + trend
season_len_da = xr.DataArray(
    season_len.values,
    coords={"time": pd.to_datetime(season_len.index.astype(str)) +
pd.offsets.YearEnd(0)},
    dims=["time"],
    name="hot_season_length"
)

plot_trend_yearly(
    season_len_da,
    title="Długość sezonu upałów (Tmax > 30°C) + trend",
    ylabel="Dni"
)
```




```
---------------------------------------------------------------------------
CoordinateValidationError                 Traceback (most recent call
last)
Cell In[50], line 32
     29 plt.show()
     31 # wykres: długość sezonu upałów + trend
---> 32 season_len_da = xr.DataArray(
     33     season_len.values,
     34     coords={"time":
pd.to_datetime(season_len.index.astype(str)) + pd.offsets.YearEnd(0)},
     35     dims=["time"],
     36     name="hot_season_length"
     37 )
     39 plot_trend_yearly(
     40     season_len_da,
     41     title="Długość sezonu upałów (Tmax > 30°C) + trend",
```

```
     42     ylabel="Dni"
     43 )

File ~\AppData\Local\Programs\Python\Python311\Lib\site-packages\
xarray\core\dataarray.py:462, in DataArray.__init__(self, data,
coords, dims, name, attrs, indexes, fastpath)
    460 data = _check_data_shape(data, coords, dims)
    461 data = as_compatible_data(data)
--> 462 coords, dims = _infer_coords_and_dims(data.shape, coords,
dims)
    463 variable = Variable(dims, data, attrs, fastpath=True)
    465 if not isinstance(coords, Coordinates):

File ~\AppData\Local\Programs\Python\Python311\Lib\site-packages\
xarray\core\dataarray.py:192, in _infer_coords_and_dims(shape, coords,
dims)
    189             var.dims = (dim,)
    190             new_coords[dim] = var.to_index_variable()
--> 192 validate_dataarray_coords(shape, new_coords, dims_tuple)
    194 return new_coords, dims_tuple

File ~\AppData\Local\Programs\Python\Python311\Lib\site-packages\
xarray\core\coordinates.py:1320, in validate_dataarray_coords(shape,
coords, dim)
   1317     invalid = any(d not in dim for d in v.dims)
   1319 if invalid:
-> 1320     raise CoordinateValidationError(
   1321         f"coordinate {k} has dimensions {v.dims}, but these "
   1322         "are not a subset of the DataArray "
   1323         f"dimensions {dim}"
   1324     )
   1326 for d, s in v.sizes.items():
   1327     if d in sizes and s != sizes[d]:

CoordinateValidationError: coordinate time has dimensions ('year',),
but these are not a subset of the DataArray dimensions ('time',)
```

```python
import numpy as np
import pandas as pd
import xarray as xr
import matplotlib.pyplot as plt
from scipy import stats

# --------------------------
# Helpery: sezon, trend, metryki zmienności
# --------------------------

SEASONS_ORDER = ["DJF", "MAM", "JJA", "SON"]

def add_season_year(da: xr.DataArray) -> xr.DataArray:
```

```
    """
    Dodaje współrzędne: season (DJF/MAM/JJA/SON) i season_year (rok
sezonu).
    DJF przypisujemy do roku stycznia/lutego (czyli grudzień idzie do
następnego roku).
    """
    if "time" not in da.dims:
        raise ValueError("DataArray musi mieć wymiar 'time'.")

    season = da["time"].dt.season

    year = da["time"].dt.year
    month = da["time"].dt.month
    season_year = xr.where((season == "DJF") & (month == 12), year +
1, year)

    da2 = da.copy()
    da2 = da2.assign_coords(season=season, season_year=season_year)
    return da2

def lin_trend(x: np.ndarray, y: np.ndarray):
    """
    Trend liniowy y = a*x + b. Zwraca: slope, intercept, p_value, r.
    """
    mask = np.isfinite(x) & np.isfinite(y)
    x2 = x[mask]
    y2 = y[mask]
    if len(x2) < 3:
        return np.nan, np.nan, np.nan, np.nan
    res = stats.linregress(x2, y2)
    return res.slope, res.intercept, res.pvalue, res.rvalue

def annual_variability(da_daily: xr.DataArray, metric="std") ->
xr.DataArray:
    """
    Liczy zmienność w skali roku z danych dziennych.
    metric: "std" albo "iqr"
    """
    grp = da_daily.groupby("time.year")
    if metric == "std":
        return grp.std("time")
    elif metric == "iqr":
        q75 = grp.quantile(0.75, dim="time")
        q25 = grp.quantile(0.25, dim="time")
        return q75 - q25
    else:
        raise ValueError("metric musi być 'std' lub 'iqr'.")

def seasonal_variability(da_daily: xr.DataArray, metric="std",
agg="mean") -> xr.DataArray:
```

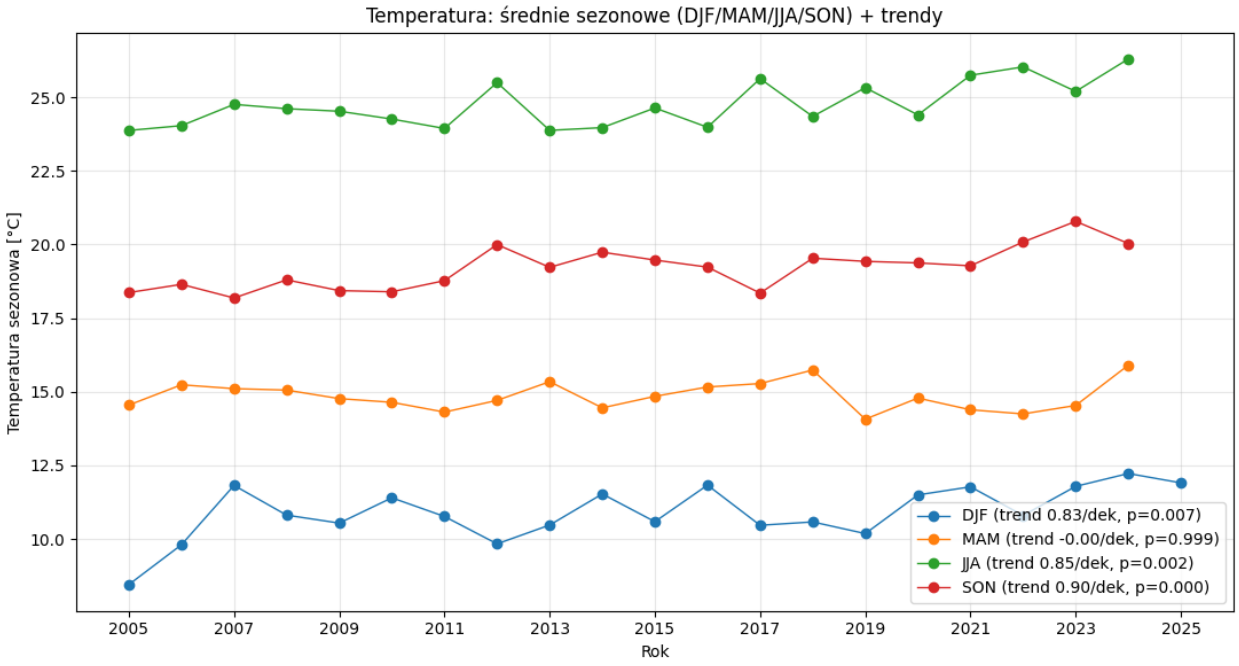
```
    """
    Zmienność w sezonach z danych dziennych.
    metric odnosi się do zmienności DZINNEJ w obrębie sezonu
(std/iqr).
    """
    da2 = add_season_year(da_daily)
    g = da2.groupby(["season_year", "season"])

    if metric == "std":
        var = g.std("time")
    elif metric == "iqr":
        q75 = g.quantile(0.75, dim="time")
        q25 = g.quantile(0.25, dim="time")
        var = q75 - q25
    else:
        raise ValueError("metric musi być 'std' lub 'iqr'.")

    var = var.sel(season=SEASONS_ORDER)
    var = var.rename({"season_year": "year"})
    # wymuś int na współrzędnej year
    var = var.assign_coords(year=var["year"].astype(int))
    return var

def seasonal_aggregate(da_daily: xr.DataArray, how="mean") ->
xr.DataArray:
    """
    Agreguje do wartości sezonowych (1 wartość na sezon na rok).
    how: "mean" (temp), "sum" (opad)
    """
    da2 = add_season_year(da_daily)
    g = da2.groupby(["season_year", "season"])

    if how == "mean":
        out = g.mean("time")
    elif how == "sum":
        out = g.sum("time")
    else:
        raise ValueError("how musi być 'mean' lub 'sum'.")

    out = out.sel(season=SEASONS_ORDER).rename({"season_year":
"year"})
    # wymuś int na współrzędnej year
    out = out.assign_coords(year=out["year"].astype(int))
    return out

def plot_seasonal_trends(seas_da: xr.DataArray, ylabel: str, title:
str, tick_step: int = 2):
    """
    Rysuje serie sezonowe + trend (osobno dla każdego sezonu).
    seas_da: dims ('year','season')
```

```
    tick_step: co ile lat podpisy na osi (1 = każdy rok, 2 = co 2
lata, itd.)
    """
    # lata jako int (do osi)
    years_int = seas_da["year"].astype(int).values
    # float do regresji (bezpiecznie)
    years = years_int.astype(float)

    plt.figure(figsize=(11, 6))
    for s in SEASONS_ORDER:
        y = seas_da.sel(season=s).values
        slope, intercept, p, r = lin_trend(years, y)
        slope_dec = slope * 10.0 if np.isfinite(slope) else np.nan

        plt.plot(years_int, y, marker="o", linewidth=1,
                 label=f"{s} (trend {slope_dec:.2f}/dek, p={p:.3f})")

    plt.title(title)
    plt.xlabel("Rok")
    plt.ylabel(ylabel)
    plt.grid(True, alpha=0.3)

    # wymuś int ticks
    if tick_step is None or tick_step <= 1:
        plt.xticks(years_int)
    else:
        plt.xticks(years_int[::tick_step])

    plt.legend()
    plt.tight_layout()
    plt.show()

def seasonal_trend_table(seas_da: xr.DataArray) -> pd.DataFrame:
    """
    Tabela trendów sezonowych (slope/decade, p, r).
    """
    years_int = seas_da["year"].astype(int).values
    years = years_int.astype(float)

    rows = []
    for s in SEASONS_ORDER:
        y = seas_da.sel(season=s).values
        slope, intercept, p, r = lin_trend(years, y)
        rows.append({
            "season": s,
            "slope_per_decade": slope * 10.0,
            "p_value": p,
            "r": r
        })
```

```
    df = (
        pd.DataFrame(rows)
        .set_index("season")
        .sort_values("slope_per_decade", ascending=False)
    )
    return df

def variability_trend_table(var_da: xr.DataArray) -> pd.DataFrame:
    """
    Tabela trendów zmienności rocznej lub sezonowej.
    var_da: może być 1D (year) albo 2D (year, season)
    """
    if "season" in var_da.dims:
        years_int = var_da["year"].astype(int).values
        years = years_int.astype(float)

        rows = []
        for s in SEASONS_ORDER:
            y = var_da.sel(season=s).values
            slope, intercept, p, r = lin_trend(years, y)
            rows.append({
                "season": s,
                "var_slope_per_decade": slope * 10.0,
                "p_value": p,
                "r": r
            })

        return (
            pd.DataFrame(rows)
            .set_index("season")
            .sort_values("var_slope_per_decade", ascending=False)
        )

    else:
        years_int = var_da["year"].astype(int).values
        years = years_int.astype(float)

        y = var_da.values
        slope, intercept, p, r = lin_trend(years, y)
        return pd.DataFrame([{
            "var_slope_per_decade": slope * 10.0,
            "p_value": p,
            "r": r
        }])


# --------------------------
# PUNKT 3: Sezonowość + „co z tego wynika” (twarde liczby do
omówienia)
# --------------------------
```

```
# Temperatura: średnia sezonowa
t2m_seas = seasonal_aggregate(t2m_daily, how="mean")
df_t2m_seas_tr = seasonal_trend_table(t2m_seas)
display(df_t2m_seas_tr)

plot_seasonal_trends(
    t2m_seas,
    ylabel="Temperatura sezonowa [°C]",
    title="Temperatura: średnie sezonowe (DJF/MAM/JJA/SON) + trendy"
)
```

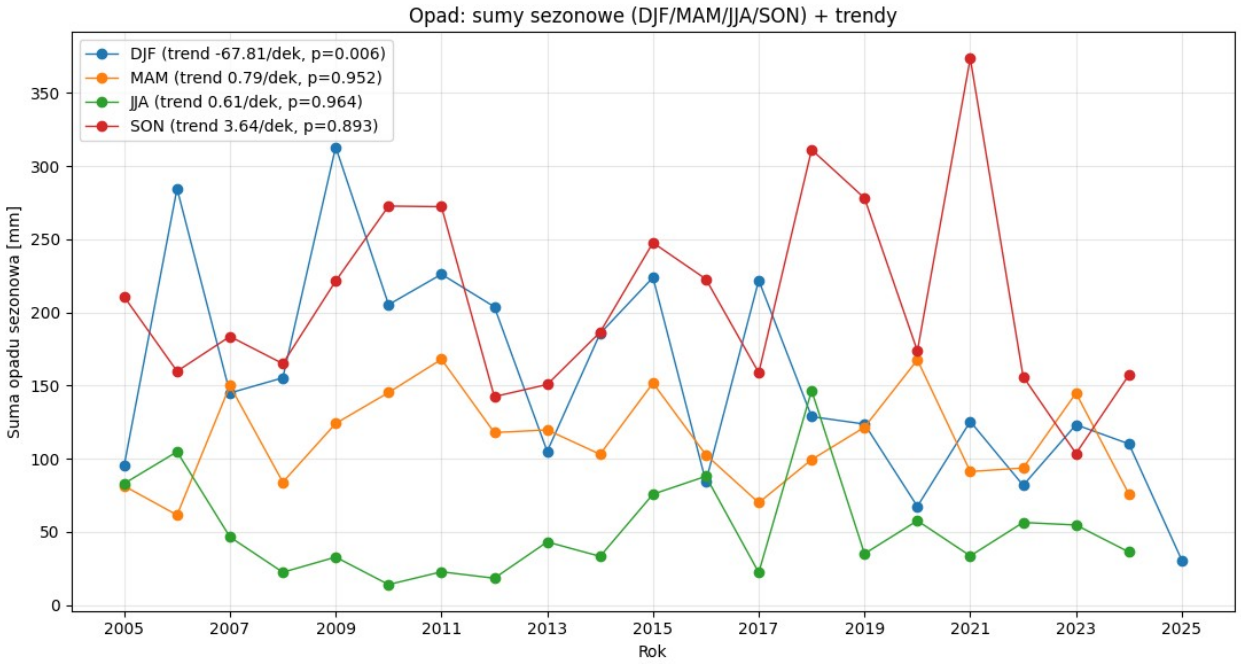
season	slope_per_decade	p_value	r
SON	0.896366	0.000144	0.749035
JJA	0.845309	0.002118	0.645391
DJF	0.831763	0.007405	0.566609
MAM	-0.000224	0.999098	-0.000270




```
# Opad: suma sezonowa
tp_seas = seasonal_aggregate(tp_daily, how="sum")
df_tp_seas_tr = seasonal_trend_table(tp_seas)
display(df_tp_seas_tr)

plot_seasonal_trends(
    tp_seas,
```

```
    ylabel="Suma opadu sezonowa [mm]",
    title="Opad: sumy sezonowe (DJF/MAM/JJA/SON) + trendy"
)
```

season	slope_per_decade	p_value	r
SON	3.635200	0.893474	0.031996
MAM	0.792588	0.951938	0.014404
JJA	0.610558	0.964173	0.010735
DJF	-67.810913	0.006380	-0.575131

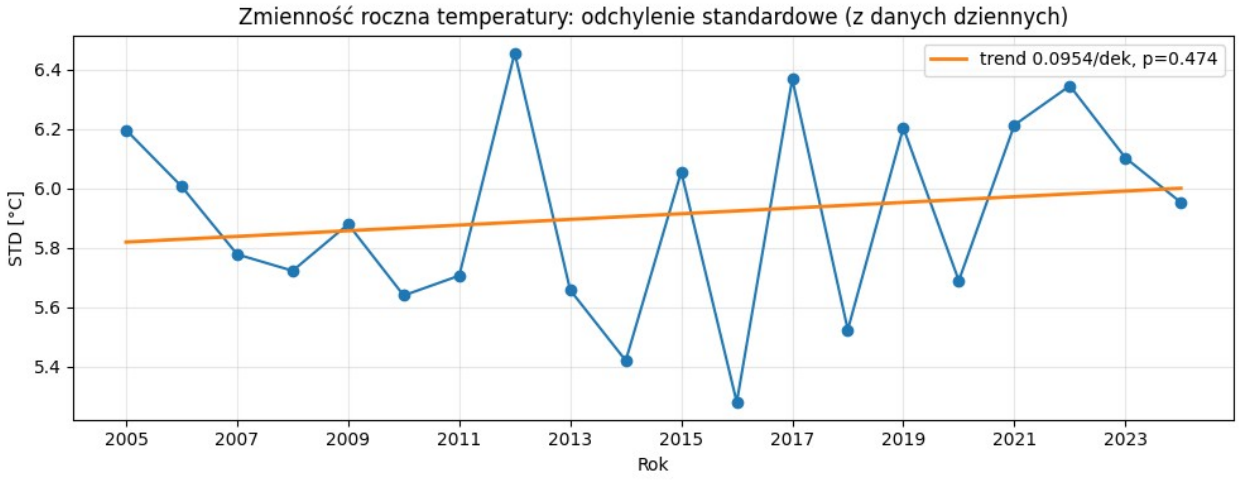


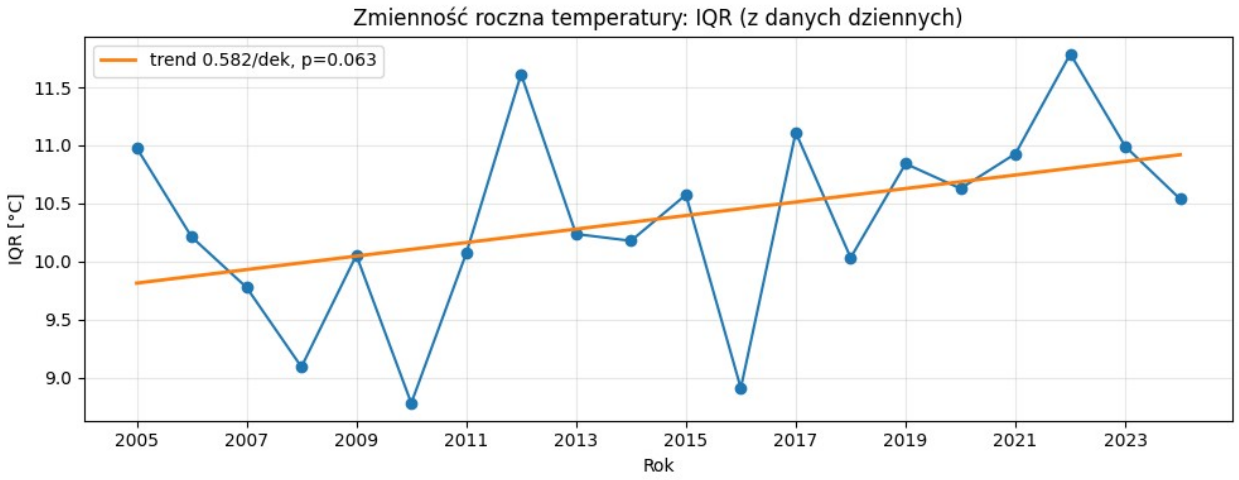

```
# --------------------------
# PUNKT 4: Zmienność i jej zmiany w czasie (STD/IQR) – rocznie + sezonowo
# --------------------------

# 4A) Zmienność roczna z danych dziennych (temperatura)
t2m_std_y = annual_variability(t2m_daily, metric="std").assign_coords(year=lambda x: x["year"].astype(int))
t2m_iqr_y = annual_variability(t2m_daily, metric="iqr").assign_coords(year=lambda x: x["year"].astype(int))

# 4B) Zmienność roczna z danych dziennych (opad)
tp_std_y = annual_variability(tp_daily, metric="std").assign_coords(year=lambda x: x["year"].astype(int))
tp_iqr_y = annual_variability(tp_daily, metric="iqr").assign_coords(year=lambda x: x["year"].astype(int))
```

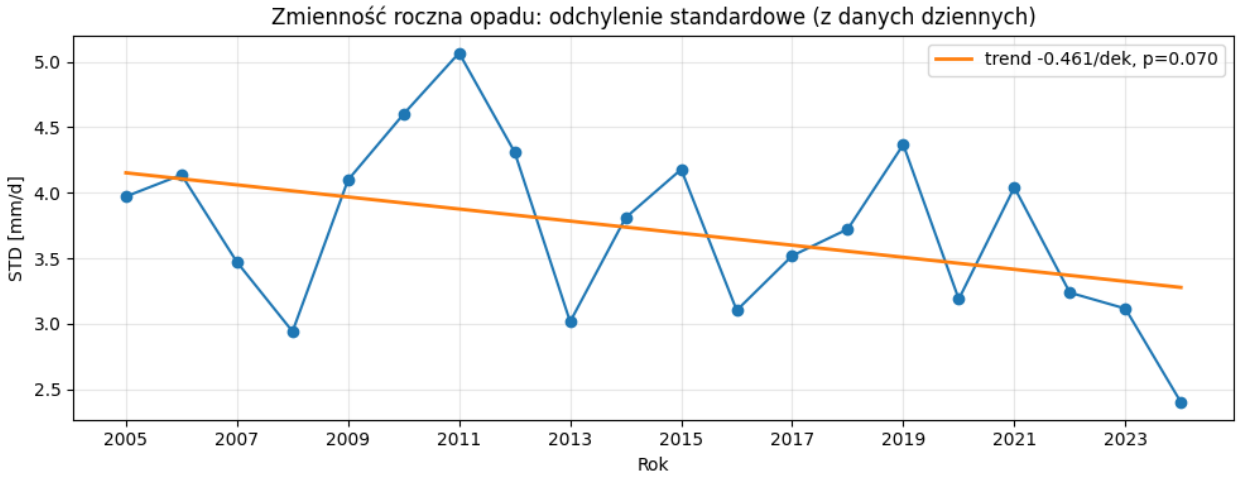
```python
# Wykresy zmienności rocznej
def plot_1d_with_trend(da_1d: xr.DataArray, ylabel: str, title: str,
tick_step: int = 2):
    years_int = da_1d["year"].astype(int).values         # do osi X
(INT)
    years = years_int.astype(float)                      # do regresji
(FLOAT)

    y = da_1d.values
    slope, intercept, p, r = lin_trend(years, y)

    plt.figure(figsize=(10,4))
    plt.plot(years_int, y, marker="o")

    if np.isfinite(slope):
        plt.plot(years_int, intercept + slope*years, linewidth=2,
                 label=f"trend {slope*10:.3g}/dek, p={p:.3f}")
        plt.legend()

    plt.title(title)
    plt.xlabel("Rok")
    plt.ylabel(ylabel)
    plt.grid(True, alpha=0.3)

    # wymuś int ticks
    if tick_step is None or tick_step <= 1:
        plt.xticks(years_int)
    else:
        plt.xticks(years_int[::tick_step])

    plt.tight_layout()
    plt.show()


plot_1d_with_trend(t2m_std_y, "STD [°C]", "Zmienność roczna
temperatury: odchylenie standardowe (z danych dziennych)",
tick_step=2)
plot_1d_with_trend(t2m_iqr_y, "IQR [°C]", "Zmienność roczna
temperatury: IQR (z danych dziennych)", tick_step=2)
plot_1d_with_trend(tp_std_y, "STD [mm/d]", "Zmienność roczna opadu:
odchylenie standardowe (z danych dziennych)", tick_step=2)
plot_1d_with_trend(tp_iqr_y, "IQR [mm/d]", "Zmienność roczna opadu:
IQR (z danych dziennych)", tick_step=2)

print("Trend zmienności rocznej (temperatura, STD):")
display(variability_trend_table(t2m_std_y))

print("Trend zmienności rocznej (opad, STD):")
display(variability_trend_table(tp_std_y))
```

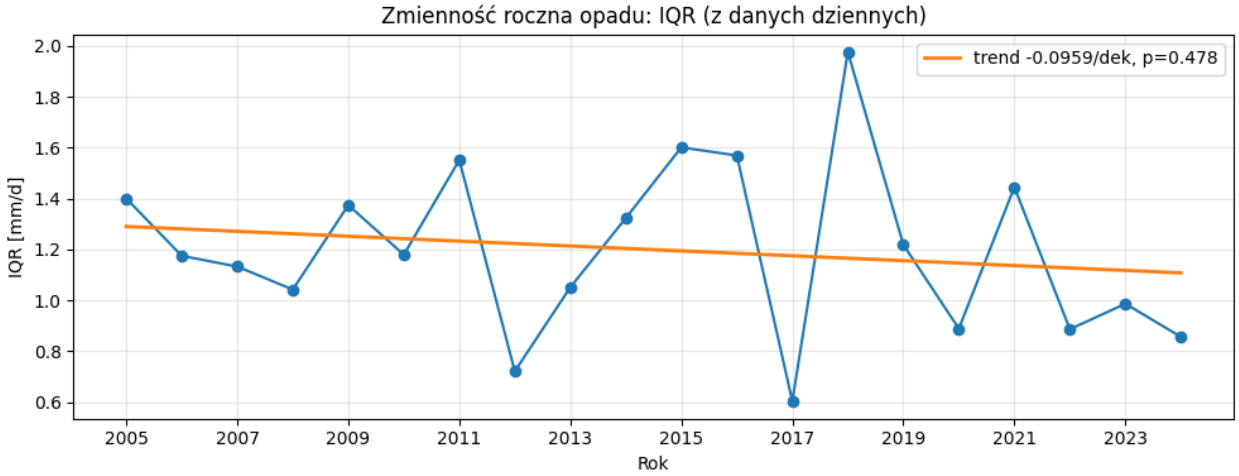
```python
# 4C) Zmienność SEZONOWA (czy np. latem rośnie rozrzut temperatur?)
t2m_std_seas = seasonal_variability(t2m_daily, metric="std")
t2m_iqr_seas = seasonal_variability(t2m_daily, metric="iqr")

tp_std_seas = seasonal_variability(tp_daily, metric="std")
tp_iqr_seas = seasonal_variability(tp_daily, metric="iqr")

# (jeśli helper seasonal_variability nie wymuszał int, to dopnijmy)
t2m_std_seas =
t2m_std_seas.assign_coords(year=t2m_std_seas["year"].astype(int))
t2m_iqr_seas =
t2m_iqr_seas.assign_coords(year=t2m_iqr_seas["year"].astype(int))
tp_std_seas  =
tp_std_seas.assign_coords(year=tp_std_seas["year"].astype(int))
tp_iqr_seas  =
tp_iqr_seas.assign_coords(year=tp_iqr_seas["year"].astype(int))

# Tabele trendów zmienności sezonowej
print("Trend zmienności sezonowej (temperatura, STD) — per sezon:")
display(variability_trend_table(t2m_std_seas))

print("Trend zmienności sezonowej (opad, STD) — per sezon:")
display(variability_trend_table(tp_std_seas))

# Wykresy zmienności sezonowej (plot_seasonal_trends masz już
poprawione na int)
plot_seasonal_trends(
    t2m_std_seas,
    ylabel="STD temperatury dziennej [°C]",
    title="Zmienność w sezonach: STD temperatury dziennej w obrębie
sezonu + trendy",
)

plot_seasonal_trends(
    tp_std_seas,
    ylabel="STD opadu dziennego [mm/d]",
    title="Zmienność w sezonach: STD opadu dziennego w obrębie sezonu
+ trendy",
)


# 4D) Dodatkowo: porównanie „wczesny” vs „późny” okres (czy zmienność
wzrosła?)
def compare_periods(da_1d: xr.DataArray, split_year=2014, label=""):
    """
    Porównuje średnią zmienność przed/po (split_year).
    """
    # upewnij się, że year jest int (dla slice też jest bezpieczniej)
```

```
    da_1d = da_1d.assign_coords(year=da_1d["year"].astype(int))

    early = da_1d.sel(year=slice(None, split_year)).values
    late  = da_1d.sel(year=slice(split_year+1, None)).values

    early_mean = np.nanmean(early)
    late_mean = np.nanmean(late)

    # test t (ostrożnie interpretować, mała próba)
    tstat, p = stats.ttest_ind(
        early[np.isfinite(early)],
        late[np.isfinite(late)],
        equal_var=False
    )

    print(f"{label} | średnia zmienność <= {split_year}:
{early_mean:.3g} | >= {split_year+1}: {late_mean:.3g} |
p(ttest)={p:.3f}")

compare_periods(t2m_std_y, split_year=2014, label="Temperatura STD
roczne")
compare_periods(tp_std_y, split_year=2014, label="Opad STD roczne")
```





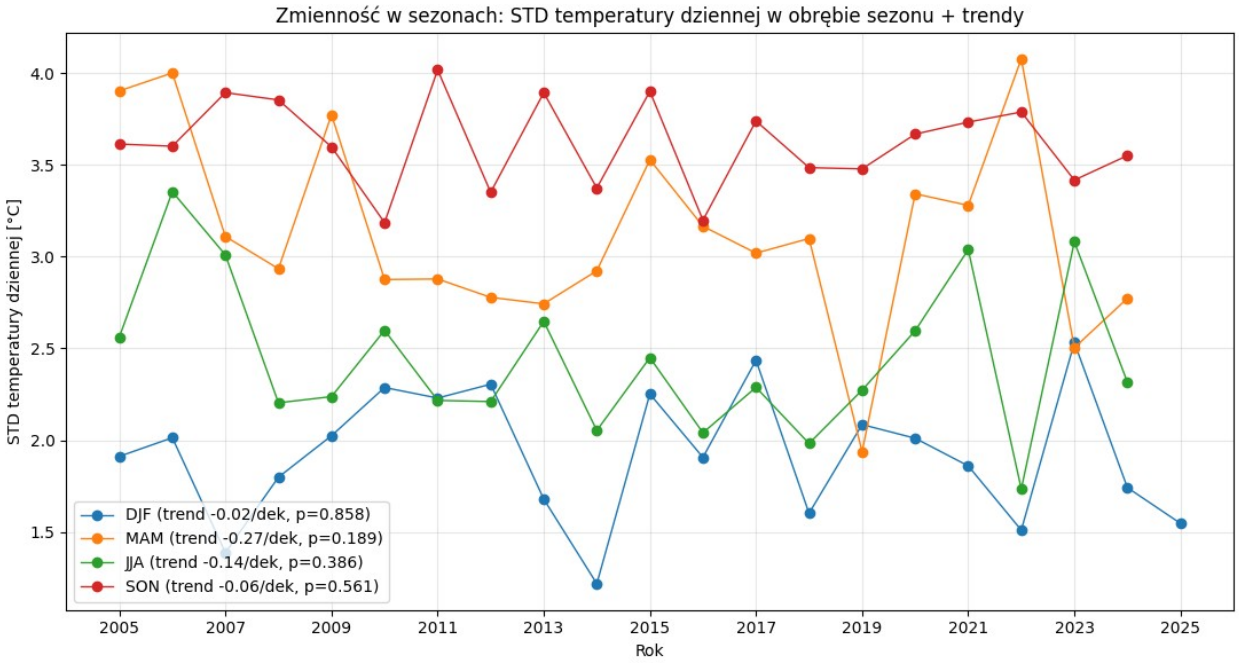


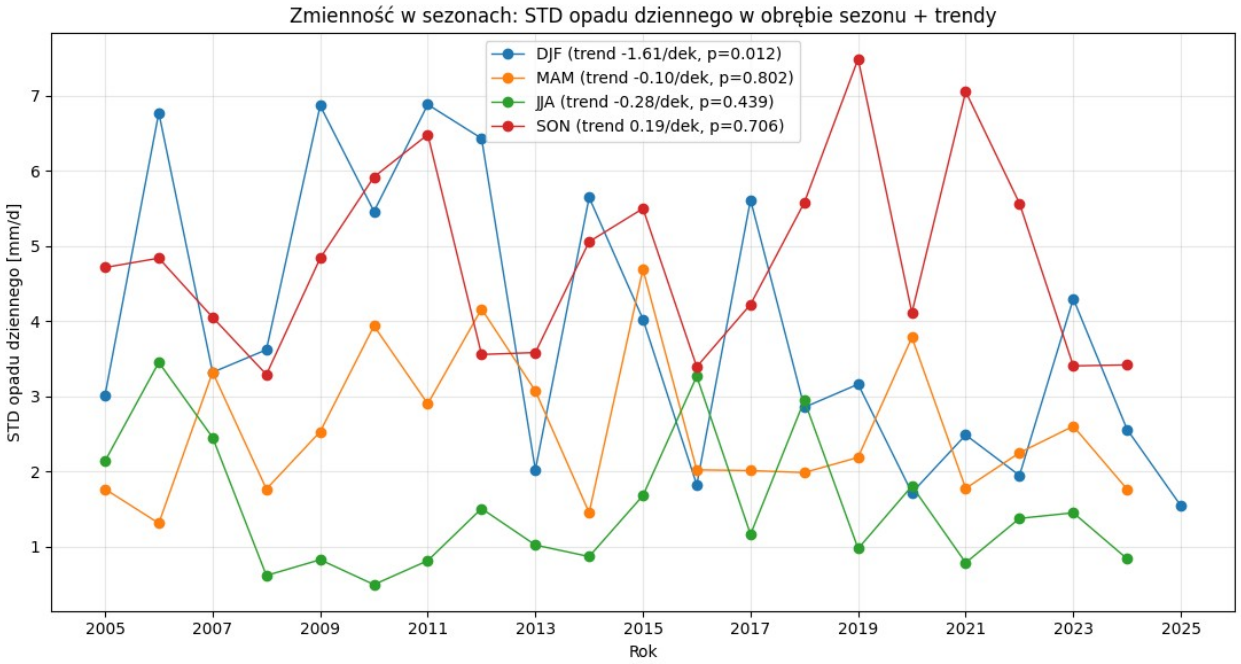


Trend zmienności rocznej (temperatura, STD):

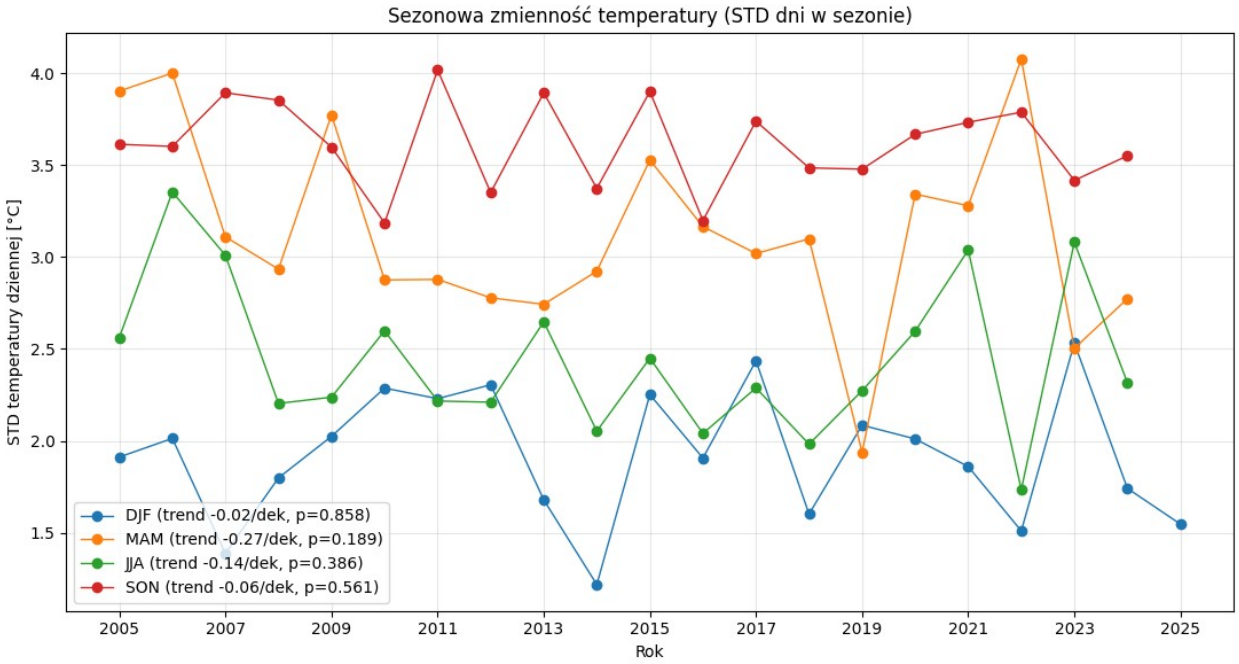
	var_slope_per_decade	p_value	r
0	0.095353	0.473728	0.169971

```
Trend zmienności rocznej (opad, STD):

   var_slope_per_decade   p_value         r
0             -0.460758  0.070353 -0.412954

Trend zmienności sezonowej (temperatura, STD) – per sezon:

        var_slope_per_decade   p_value         r
season
DJF                -0.023538  0.858192 -0.041516
SON                -0.055795  0.560986 -0.138277
JJA                -0.144641  0.386260 -0.204861
MAM                -0.274067  0.189155 -0.306218

Trend zmienności sezonowej (opad, STD) – per sezon:

        var_slope_per_decade   p_value         r
season
SON                 0.193407  0.705862  0.090018
MAM                -0.098078  0.802480 -0.059729
JJA                -0.277095  0.438688 -0.183501
DJF                -1.608687  0.012475 -0.534895
```

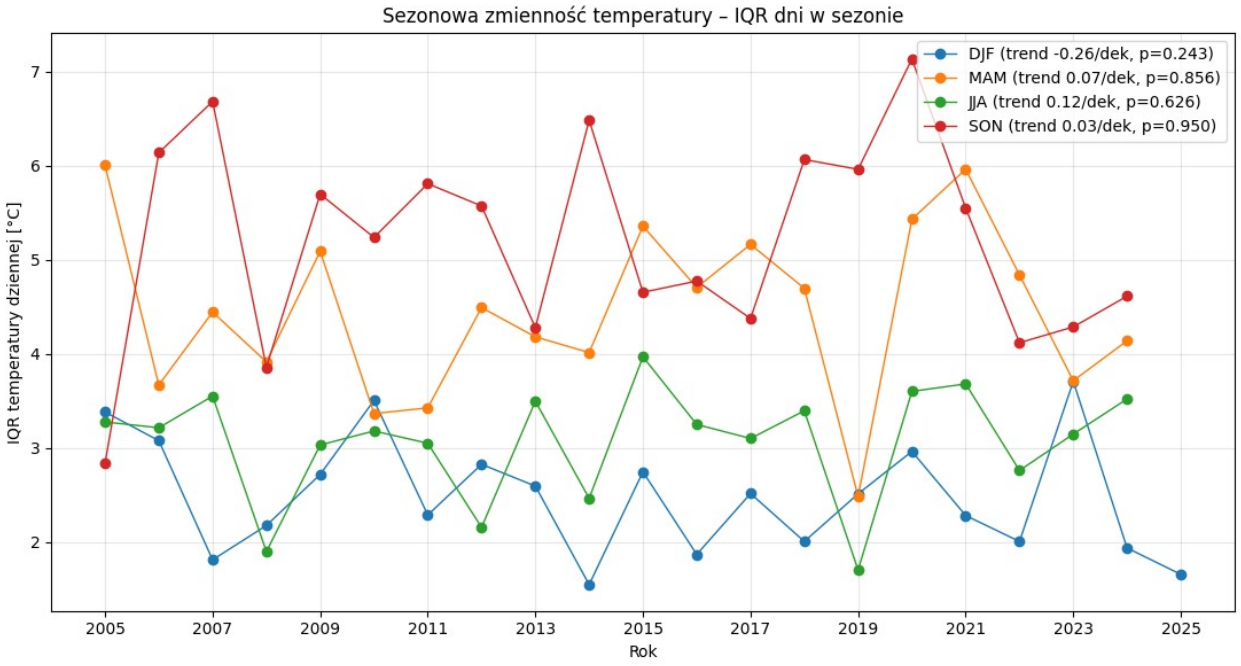




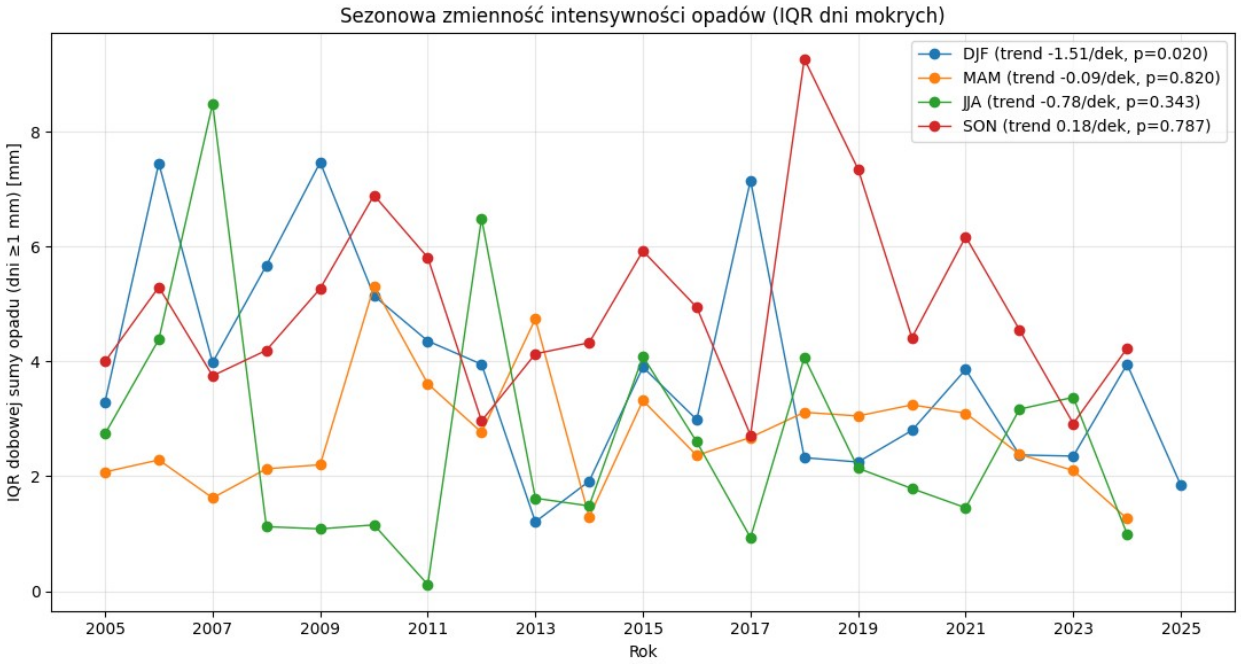

```
Temperatura STD roczne | średnia zmienność <= 2014: 5.85 | >= 2015:
5.97 | p(ttest)=0.406
Opad STD roczne | średnia zmienność <= 2014: 3.94 | >= 2015: 3.49 |
p(ttest)=0.126

da2 = add_season_year(t2m_daily)

t2m_seasonal_std = (
    da2
    .groupby(["season_year", "season"])
    .std("time")
    .rename({"season_year": "year"})
)

# Sezonowa zmienność temperatury (STD dni)
t2m_std_season = (
    add_season_year(t2m_daily)
    .groupby(["season_year", "season"])
    .std("time")
    .rename({"season_year": "year"})
    .assign_coords(year=lambda x: x["year"].astype(int))
)


plot_seasonal_trends(
    t2m_std_season,
    ylabel="STD temperatury dziennej [°C]",
    title="Sezonowa zmienność temperatury (STD dni w sezonie)"
)
```



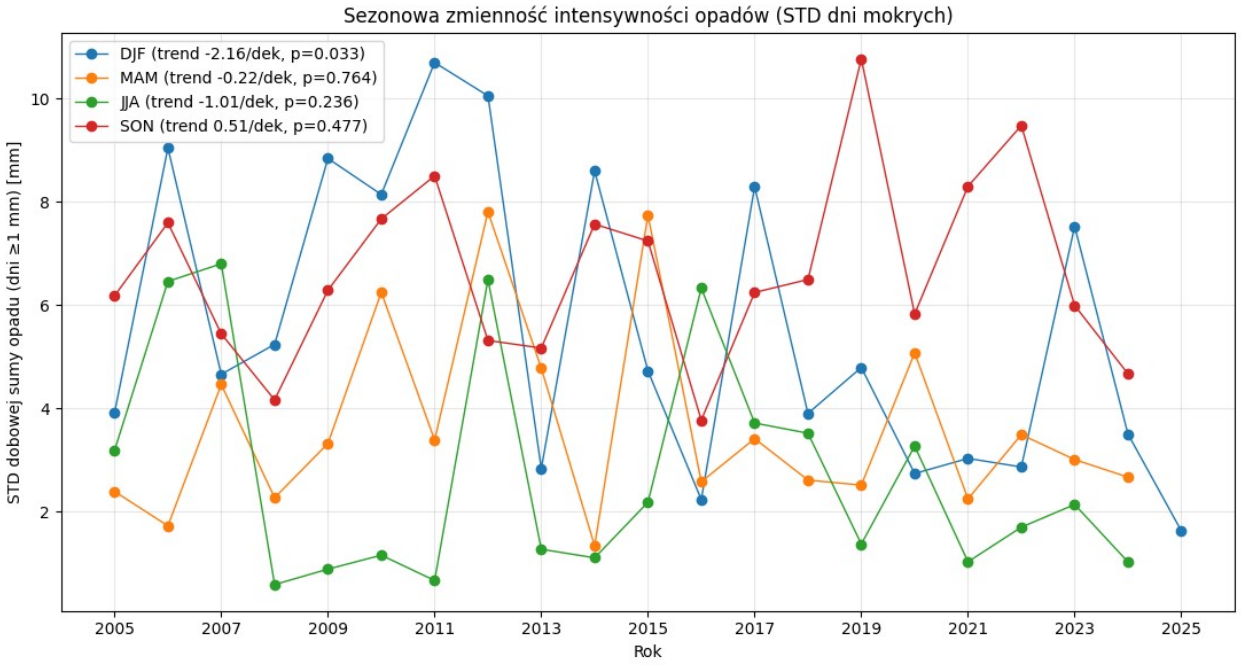

```
#IQR sezonowe temperatury (dni)

t2m_iqr_season = (
    add_season_year(t2m_daily)
    .groupby(["season_year", "season"])
    .quantile(0.75, dim="time")
    - add_season_year(t2m_daily)
      .groupby(["season_year", "season"])
      .quantile(0.25, dim="time")
)

t2m_iqr_season = (
    t2m_iqr_season
    .rename({"season_year": "year"})
    .sel(season=SEASONS_ORDER)
    .assign_coords(year=lambda x: x["year"].astype(int))
)

plot_seasonal_trends(
    t2m_iqr_season,
    ylabel="IQR temperatury dziennej [°C]",
    title="Sezonowa zmienność temperatury – IQR dni w sezonie"
)
```




```
# Sezonowa zmienność opadów IQR dla dni mokrych ---

tp_wet = tp_daily.where(tp_daily >= 1.0)

tp_iqr_season = (
    add_season_year(tp_wet)
    .groupby(["season_year", "season"])
    .quantile(0.75, dim="time")
    - add_season_year(tp_wet)
      .groupby(["season_year", "season"])
      .quantile(0.25, dim="time")
)

tp_iqr_season = (
    tp_iqr_season
    .rename({"season_year": "year"})
    .sel(season=SEASONS_ORDER)
    .assign_coords(year=lambda x: x["year"].astype(int))
)

plot_seasonal_trends(
    tp_iqr_season,
    ylabel="IQR dobowej sumy opadu (dni ≥1 mm) [mm]",
    title="Sezonowa zmienność intensywności opadów (IQR dni mokrych)"
)
```




```
#Sezonowa zmienność opadów STD dla dni mokrych

tp_wet = tp_daily.where(tp_daily >= 1.0)


tp_std_season = (
    add_season_year(tp_wet)
    .groupby(["season_year", "season"])
    .std("time")
    .rename({"season_year": "year"})
    .sel(season=SEASONS_ORDER)
    .assign_coords(year=lambda x: x["year"].astype(int))
)

plot_seasonal_trends(
    tp_std_season,
    ylabel="STD dobowej sumy opadu (dni ≥1 mm) [mm]",
    title="Sezonowa zmienność intensywności opadów (STD dni mokrych)"
)
```




```
t2m_box = t2m.sel(latitude=lat_slice, longitude=lon_slice)

# dobowa średnia temperatury dla każdego punktu siatki 3x3
t2m_daily_grid = t2m_box.resample(time="1D").mean()

tp_acc_box = ds_opad.sel(latitude=lat_slice, longitude=lon_slice)

# przyrost godzinowy (eliminuje akumulację)
tp_inc = tp_acc_box.diff("time")

# odfiltruj ujemne (reset akumulacji)
tp_inc = tp_inc.where(tp_inc >= 0)

# dobowa suma opadu dla każdego punktu siatki 3x3
tp_daily_grid = tp_inc.resample(time="1D").sum()
```

```
ETNA_LAT = 37.7522
ETNA_LON = 14.9952

def haversine_km(lat1, lon1, lat2, lon2):
    R = 6371.0
    lat1 = np.deg2rad(lat1); lon1 = np.deg2rad(lon1)
    lat2 = np.deg2rad(lat2); lon2 = np.deg2rad(lon2)
    dlat = lat2 - lat1
    dlon = lon2 - lon1
    a = np.sin(dlat/2)**2 +
np.cos(lat1)*np.cos(lat2)*np.sin(dlon/2)**2
    return 2*R*np.arcsin(np.sqrt(a))

lat2d, lon2d = xr.broadcast(t2m_daily_grid["latitude"],
t2m_daily_grid["longitude"])
dist_km = xr.apply_ufunc(
    haversine_km,
    lat2d, lon2d,
    xr.full_like(lat2d, ETNA_LAT),
    xr.full_like(lon2d, ETNA_LON),
    vectorize=True,
    output_dtypes=[float],
)

# spłaszcz punkty 3×3 do wymiaru "point"
dist_p =
dist_km.stack(point=("latitude","longitude")).sortby(dist_km.stack(poi
nt=("latitude","longitude")))

near_pts = dist_p["point"].values[:2]
far_pts  = dist_p["point"].values[-2:]

# przygotuj dzienne serie "near" i "far" dla temperatury i opadu
t2m_p = t2m_daily_grid.stack(point=("latitude","longitude"))
tp_p = tp_daily_grid["tp"].stack(point=("latitude","longitude"))

t2m_near = t2m_p.sel(point=near_pts).mean("point")
t2m_far  = t2m_p.sel(point=far_pts ).mean("point")

tp_near  = tp_p.sel(point=near_pts).mean("point")
tp_far   = tp_p.sel(point=far_pts ).mean("point")

# różnica (near - far)
t2m_diff = t2m_near - t2m_far
tp_diff  = tp_near  - tp_far

print("Średnia różnica dzienna (near - far):")
print("t2m [°C]:", float(t2m_diff.mean("time")))
print(
    "tp [mm/d]:",
```

```
    float(tp_diff["tp"].mean("time").values)
)

Średnia różnica dzienna (near - far):
t2m [°C]: -3.597447156906128

---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call
last)
File ~\AppData\Local\Programs\Python\Python311\Lib\site-packages\
xarray\core\dataarray.py:876, in DataArray._getitem_coord(self, key)
    875 try:
--> 876     var = self._coords[key]
    877 except KeyError:

KeyError: 'tp'

During handling of the above exception, another exception occurred:

KeyError                                  Traceback (most recent call
last)
Cell In[220], line 47
     43 print("Średnia różnica dzienna (near - far):")
     44 print("t2m [°C]:", float(t2m_diff.mean("time")))
     45 print(
     46     "tp [mm/d]:",
---> 47     float(tp_diff["tp"].mean("time").values)
     48 )

File ~\AppData\Local\Programs\Python\Python311\Lib\site-packages\
xarray\core\dataarray.py:885, in DataArray.__getitem__(self, key)
    883 def __getitem__(self, key: Any) -> Self:
    884     if isinstance(key, str):
--> 885         return self._getitem_coord(key)
    886     else:
    887         # xarray-style array indexing
    888         return self.isel(indexers=self._item_key_to_dict(key))

File ~\AppData\Local\Programs\Python\Python311\Lib\site-packages\
xarray\core\dataarray.py:879, in DataArray._getitem_coord(self, key)
    877 except KeyError:
    878     dim_sizes = dict(zip(self.dims, self.shape, strict=True))
--> 879     _, key, var = _get_virtual_variable(self._coords, key,
dim_sizes)
    881 return self._replace_maybe_drop_dims(var, name=key)

File ~\AppData\Local\Programs\Python\Python311\Lib\site-packages\
xarray\core\dataset_utils.py:79, in _get_virtual_variable(variables,
key, dim_sizes)
```

```
     77 split_key = key.split(".", 1)
     78 if len(split_key) != 2:
---> 79     raise KeyError(key)
     81 ref_name, var_name = split_key
     82 ref_var = variables[ref_name]

KeyError: 'tp'
```

```
wet = tp_daily_grid >= 1.0
tp_wet = tp_daily_grid.where(wet)

p95 = tp_wet.quantile(0.95, dim="time", skipna=True)
p99 = tp_wet.quantile(0.99, dim="time", skipna=True)

# liczba dni ekstremalnych w roku dla każdego punktu 3×3
ext95_days_year = (tp_daily_grid > p95).resample(time="YE").sum()
ext99_days_year = (tp_daily_grid > p99).resample(time="YE").sum()
```

```
C:\Users\kajas\AppData\Local\Programs\Python\Python311\Lib\site-
packages\numpy\lib\nanfunctions.py:1563: RuntimeWarning: All-NaN slice
encountered
  return function_base._ureduce(a,
```

```
print(t2m_box.dims, t2m_box.sizes)
print(tp_acc_box.dims, tp_acc_box.sizes)
```

```
('time', 'latitude', 'longitude') Frozen({'time': 175320, 'latitude':
3, 'longitude': 3})
FrozenMappingWarningOnValuesAccess({'time': 14614, 'step': 12,
'latitude': 3, 'longitude': 3}) Frozen({'time': 14614, 'step': 12,
'latitude': 3, 'longitude': 3})
```